

A Generalized Algorithm for Multi-Objective Reinforcement Learning and Policy Adaptation

Runzhe Yang

Department of Computer Science
Princeton University
runzhey@cs.princeton.edu

Xingyuan Sun

Department of Computer Science
Princeton University
xs5@cs.princeton.edu

Karthik Narasimhan

Department of Computer Science
Princeton University
karthikn@cs.princeton.edu

Abstract

We introduce a new algorithm for multi-objective reinforcement learning (MORL) with linear preferences, with the goal of enabling few-shot adaptation to new tasks. In MORL, the aim is to learn policies over multiple competing objectives whose relative importance (*preferences*) is unknown to the agent. While this alleviates dependence on scalar reward design, the expected return of a policy can change significantly with varying preferences, making it challenging to learn a single model to produce optimal policies under different preference conditions. We propose a generalized version of the Bellman equation to learn a single parametric representation for optimal policies over the space of all possible preferences. After an initial learning phase, our agent can execute the optimal policy under any given preference, or automatically infer an underlying preference with very few samples. Experiments across four different domains demonstrate the effectiveness of our approach.¹

1 Introduction

In recent years, there has been increased interest in the paradigm of multi-objective reinforcement learning (MORL), which deals with learning control policies to simultaneously optimize over several criteria. Compared to traditional RL, where the aim is to optimize for a scalar reward, the optimal policy in a multi-objective setting depends on the relative preferences among competing criteria. For example, consider a virtual assistant (Figure 1) that can communicate with a human to perform a specific task (e.g., provide weather or navigation information). Depending on the user’s relative preferences between aspects like success rate or brevity, the agent might need to follow completely different strategies. If success is all that matters (e.g., providing an accurate weather report), the agent might provide detailed responses or ask several follow-up questions. On the other hand, if brevity is crucial (e.g., while providing

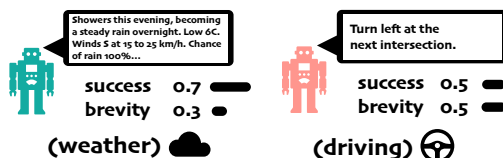


Figure 1: Task-oriented dialogue policy learning is a real-life example of unknown linear preference scenario. Users may expect either briefer dialogue or more informative dialogue depending on the task.

¹Code is available at <https://github.com/RunzheYang/MORL>

turn-by-turn guidance), the agent needs to find the shortest way to complete the task. In traditional RL, this is often a fixed choice made by the designer and incorporated into the scalar reward. While this suffices in cases where we know the preferences of a task beforehand, the learned policy is limited in its applicability to scenarios with different preferences. The MORL framework provides two distinct advantages – (1) reduced dependence on scalar reward design to combine different objectives, which is both a tedious manual task and can lead to unintended consequences [1], and (2) dynamic adaptation or transfer to related tasks with different preferences.

However, learning policies over multiple preferences under the MORL setting has proven to be quite challenging, with most prior work using one of two strategies [2]. The first is to convert the multi-objective problem into a single-objective one through various techniques [3–6] and use traditional RL algorithms. These methods only learn an ‘average’ policy over the space of preferences and cannot be tailored to be optimal for specific preferences. The second strategy is to compute a set of optimal policies that encompass the entire space of possible preferences in the domain [7–9]. The main drawback of these approaches is their lack of scalability – the challenge of representing a Pareto front (or its convex approximation) of optimal policies is handled by learning several individual policies, which can grow significantly with the size of the domain.

In this paper, we propose a novel algorithm for learning a single policy network that is optimized over the entire space of preferences in a domain. This allows our trained model to produce the optimal policy for any user-specified preference. We tackle two concrete challenges in MORL: (1) provide theoretical convergence results of a multi-objective version of Q-Learning for MORL with linear preferences, and (2) demonstrate effective use of deep neural networks to scale MORL to larger domains. Our algorithm is based on two key insights – (1) the optimality operator for a generalized version of Bellman equation [10] with preferences is a valid contraction, and (2) optimizing for the convex envelope of multi-objective Q-values ensures an efficient alignment between preferences and corresponding optimal policies. We use hindsight experience replay [11] to re-use transitions for learning with different sampled preferences and homotopy optimization [12] to ensure tractable learning. In addition, we also demonstrate how to use our trained model to automatically infer hidden preferences on a new task, when provided with just scalar rewards, through a combination of policy gradient and stochastic search over the preference parameters.

We perform empirical evaluation on four different domains – deep sea treasure (a popular MORL benchmark), a fruit tree navigation task, task-oriented dialog, and the video game Super Mario Bros. Our experiments demonstrate that our methods significantly outperform competitive baselines on all domains. For instance, our envelope MORL algorithm achieves an % improvement on average user utility compared to the scalarized MORL in the dialog task and a factor 2x average improvement on SuperMario game with random preferences. We also demonstrate that our agent can reasonably infer hidden preferences at test time using very few sampled trajectories.

2 Background

A multi-objective Markov decision process (MOMDP) can be represented by the tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathbf{r}, \Omega, f_\Omega \rangle$ with state space $\mathcal{S}$, action space $\mathcal{A}$, transition distribution $\mathcal{P}(s'|s, a)$, vector reward function $\mathbf{r}(s, a)$, the space of preferences Ω , and preference functions, e.g., $f_\omega(\mathbf{r})$ which produces a scalar utility using preference $\omega \in \Omega$. In this work, we consider the class of MOMDPs with linear preference functions, i.e., $f_\omega(\mathbf{r}(s, a)) = \omega^\top \mathbf{r}(s, a)$. We observe that if ω is fixed to a single value, this MOMDP collapses into a standard MDP. On the other hand, if we consider all possible returns from an MOMDP, we have a Pareto frontier $\mathcal{F}^* := \{\hat{\mathbf{r}} \mid \nexists \hat{\mathbf{r}}' \geq \hat{\mathbf{r}}\}$, where the return $\hat{\mathbf{r}} := \sum_t \gamma^t \mathbf{r}(s_t, a_t)$. And for all possible preference in Ω , we define a convex coverage set (CCS) of the Pareto frontier as:

$$\text{CCS} := \{\hat{\mathbf{r}} \in \mathcal{F}^* \mid \exists \omega \in \Omega \text{ s.t. } \omega^\top \hat{\mathbf{r}} \geq \omega^\top \hat{\mathbf{r}}', \forall \hat{\mathbf{r}}' \in \mathcal{F}^*\},$$

which contains all returns that provide the maximum cumulative utility. Figure 2 (a) shows an example of CCS and the Pareto frontier. The CCS is a subset of the Pareto frontier (points A to H, and K), containing all the solutions on its outer convex boundary (excluding point K). When a specific linear preference ω is given, the point within the CCS with the largest projection along the direction of the relative importance weights will be the optimal solution (Figure 2(b)).

Our goal is to train an agent to recover policies for the entire CCS of MOMDP and then adapt to the optimal policy for any given $\omega \in \Omega$ at test time. We emphasize that we are not solving for a single,

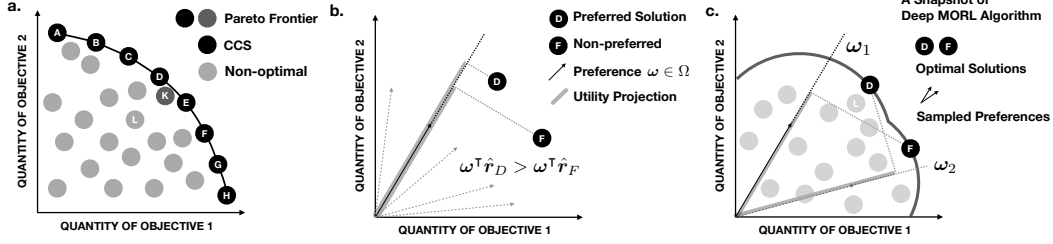


Figure 2: (a) The Pareto frontier may encapsulate local concave parts (points A-H, plus point K), whereas CCS is a convex subset of Pareto frontier (points A-H). Point L indicates a non-optimal solution. (b) Linear preferences select the optimal solution from CCS with the highest utility, represented by the projection length along preference vector. Arrows are different linear preferences, and points indicate possible returns. Return D has better cumulative utility than return F under the preference in solid line. (c) The scalarized MORL algorithms (e.g., [13]) find the optimal solutions at a stage while they are not aligned with preference, e.g., two optimal solutions D and F in the CCS, misaligned with preferences ω_2 and ω_1 . The scalarized update cannot use the information of $\max_a Q(s, a, \omega_1)$ (corresponding to F) to update the optimal solution aligned with ω_2 or vice versa. It only searches along ω_1 direction leading to non-optimal L, even if solution D has been seen under ω_2 . It still requires many iterations for the value-preference alignment.

unknown ω , but instead aim for generalization across the entire space of preferences. Accordingly, our MORL setup has two phases:

Learning phase. In this phase, the agent learns a set of *optimal policies* $\Pi_{\mathcal{L}}$ corresponding to the entire CCS of the MOMDP, using interactions with the environment and historical trajectories. For each $\pi \in \Pi_{\mathcal{L}}$, there exists at least one linear preference ω such that no other policy π' generates higher utility under that ω :

$$\pi \in \Pi_{\mathcal{L}} \Rightarrow \exists \omega \in \Omega, \text{ s.t. } \forall \pi' \in \Pi, \omega^T v^\pi(s_0) \geq \omega^T v^{\pi'}(s_0),$$

where s_0 is a fixed initial state, and v^π is the value function, i.e., $v^\pi(s) = \mathbb{E}_\pi[\hat{r}|s_0 = s]$. Given any preference ω , $\Pi_{\mathcal{L}}(\omega)$ determines the optimal policy.

Adaptation phase. After learning, the agent is provided a new task, with either a) a preference ω specified by a human, or b) an unknown preference, where the agent has to automatically infer ω . Efficiently aligning $\Pi_{\mathcal{L}}(\omega)$ with the preferred optimal policy is non-trivial since the CCS can be very large. In both cases, the agent is evaluated on how well it can adapt to tasks with unseen preferences.

2.1 Related Work

Multi-Objective RL. Existing MORL algorithms can be roughly divided into two main categories [14, 15, 2]: *single-policy methods* and *multiple-policy methods*. Single-policy methods aim to find the optimal policy for a given preference among the objectives [16, 17]. These methods explore different forms of preference functions, including non-linear ones such as the minimum over all objectives or the number of objectives that exceed a certain threshold. However, *single-policy methods do not work when preferences are unknown*.

Multi-policy approaches learn a set of policies to obtain the approximate Pareto frontier of optimal solutions. The most common strategy is to perform multiple runs of a single-policy method over different preferences [7, 18]. Policy-based RL algorithms [19, 20] simultaneously learn the optimal manifold over a set of preferences. Several value-based reinforcement learning algorithms employ an extended version of the Bellman equation and maintain the convex hull of the discrete Pareto frontier [8, 21, 22]. Multi-objective fitted Q-iteration (MOFQI) [23, 24] encapsulates preferences as input to a Q-function approximator and uses expanded historical trajectories to learn multiple policies. This allows the agent to construct the optimal policy for any given preference during testing. However, these methods explicitly maintain sets of policies, and hence *are difficult to scale up to high-dimensional preference spaces*. Furthermore, these methods are designed to work during the learning phase but cannot be easily adapted to new preferences at test time.

Scalarized Q-Learning. Recent work has proposed the scalarized Q-learning algorithm [9] which uses a vector value function but performs updates after computing the inner product of the value function with a preference vector. This method uses an outer loop to perform a search over preferences,

while the inner loop performs the scalarized updates. Recently, Abels et al. [13] extended this to use a single neural network to represent value functions over the entire space of preferences. However, scalarized updates are not sample efficient and lead to sub-optimal MORL policies – our approach uses a global optimality filter to perform envelope Q-function updates, leading to faster and better learning (as we demonstrate in Figure 2(c) and Section 4).

Three key contributions distinguish our work from Abels et al. [13]: (1) At algorithmic level, our envelope Q-learning algorithm utilizes the convex envelope of the solution frontier to update parameters of the policy network, which allows our method to quickly align one preference with optimal rewards and trajectories that may have been explored under other preferences. (2) At theoretical level, we introduce a theoretical framework for designing and analyzing value-based MORL algorithms, and convergence proofs for our envelope Q-learning algorithm. (3) At empirical level, we provide new evaluation metrics and benchmark environments for MORL and apply our algorithm to a wider variety of domains including two complex larger scale domains – task-oriented dialog and supermario. Our FTN domain is a scaled up, more complex version of Minecart in [13].

Policy Adaptation. Our policy adaptation scheme is related to prior work in preference elicitation [25–27] or inverse reinforcement learning [28, 29]. Inverse RL (IRL) aims to learn a scalar reward function from expert demonstrations, or directly imitate the expert’s policy without intermediate steps for solving a scalar reward function [30]. Chajewska et al. [31] proposed a Bayesian version to learn the utility function. IRL is effective when the hidden preference is fixed and expert demonstrations are available. In contrast, we require policy adaptation across various different preferences and do not use any demonstrations.

3 Multi-objective RL with Envelope Value Updates

In this section, we propose a new algorithm for multi-objective RL called *envelope Q-learning*. Our key idea is to use vectorized value functions and perform *envelope updates*, which utilize the convex envelope of the solution frontier to update parameters. This is in contrast to approaches like *scalarized Q-Learning*, which perform value function updates using only a single preference at a time. Since we learn a set of policies simultaneously over multiple preferences, and our concept of optimality is defined on vectorized rewards, existing convergence results from single-objective RL no longer hold. Hence, we first provide a theoretical analysis of our proposed update scheme below followed by a sketch of the resulting algorithm.

Bellman operators. The standard Q-Learning [32] algorithm for single-objective RL utilizes the Bellman optimality operator T :

$$(TQ)(s, a) := r(s, a) + \gamma \mathbb{E}_{s' \sim \mathcal{P}(\cdot|s, a)}(HQ)(s'). \quad (1)$$

where the operator H is defined by $(HQ)(s') := \sup_{a' \in \mathcal{A}} Q(s', a')$ is an optimality filter over the Q-values for the next state s' .

We extend this to the MORL case by considering a value space $\mathcal{Q} \subseteq (\Omega \rightarrow \mathbb{R}^m)^{\mathcal{S} \times \mathcal{A}}$, containing all bounded functions $Q(s, a, \omega)$ – estimates of expected total rewards under m -dimensional preference (ω) vectors. We can define a corresponding value metric d as:

$$d(Q, Q') := \sup_{\substack{s \in \mathcal{S}, a \in \mathcal{A} \\ \omega \in \Omega}} |\omega^\top (Q(s, a, \omega) - Q'(s, a, \omega))|. \quad (2)$$

Since the identity of indiscernibles [33] does not hold, we note that d forms a complete pseudo-metric space, and refer to Q as a *Multi-Objective Q-value (MOQ) function*. Given a policy π and sampled trajectories τ , we first define a multi-objective evaluation operator $\mathcal{T}_\pi$ as:

$$(\mathcal{T}_\pi Q)(s, a, \omega) := r(s, a) + \gamma \mathbb{E}_{\tau \sim (\mathcal{P}, \pi)} Q(s', a', \omega). \quad (3)$$

We then define an optimality filter $\mathcal{H}$ for the MOQ function as $(\mathcal{H}Q)(s, \omega) := \arg_Q \sup_{a \in \mathcal{A}, \omega' \in \Omega} \omega^\top Q(s, a, \omega')$, where the $\arg_Q$ takes the multi-objective value corresponding to the supremum (i.e., $Q(s, a, \omega')$ such that $(a, \omega') \in \arg \sup_{a \in \mathcal{A}, \omega' \in \Omega} \omega^\top Q(s, a, \omega')$). The return of $\arg_Q$ depends on which ω is chosen for scalarization, and we keep $\arg_Q$ for simplicity. This can be thought of as generalized version of the single-objective optimality filter in Eq. 1. Intuitively, $\mathcal{H}$ solves the convex envelope (hence the name envelope Q-learning) of the current solution frontier to

produce the Q that optimizes utility given state s and preference ω . This allows for more optimistic Q -updates compared to using just the standard Bellman filter (H) that optimizes over actions only – this is the update used by scalarized Q -learning [13]. We can then define a *multi-objective optimality operator* $\mathcal{T}$ as:

$$(\mathcal{T}Q)(s, a, \omega) := r(s, a) + \gamma \mathbb{E}_{s' \sim \mathcal{P}(\cdot|s, a)}(\mathcal{H}Q)(s', \omega). \quad (4)$$

The following theorems demonstrate the feasibility of using our optimality operator for multi-objective RL. Proofs for all the theorems are provided in the supplementary material.

Theorem 1 (Fixed Point of Envelope Optimality Operator). *Let $Q^* \in \mathcal{Q}$ be the preferred optimal value function in the value space, such that*

$$Q^*(s, a, \omega) = \arg_Q \sup_{\pi \in \Pi} \omega^\top \mathbb{E}_{\tau \sim (\mathcal{P}, \pi)} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right], \quad (5)$$

where the $\arg_Q$ takes the multi-objective value corresponding to the supremum. Then, $Q^* = \mathcal{T}Q^*$.

Theorem 1 tells us the preferred optimal value function is a fixed-point of $\mathcal{T}$ in the value space.

Theorem 2 (Envelope Optimality Operator is a Contraction). *Let Q, Q' be any two multi-objective Q -value functions in the value space $\mathcal{Q}$ as defined above. Then, the Lipschitz condition $d(\mathcal{T}Q, \mathcal{T}Q') \leq \gamma d(Q, Q')$ holds, where $\gamma \in [0, 1)$ is the discount factor of the underlying MOMDP $\mathcal{M}$.*

Finally, we provide a generalized version of Banach’s Fixed-Point Theorem in the pseudo-metric space.

Theorem 3 (Multi-Objective Banach Fixed-Point Theorem). *If $\mathcal{T}$ is a contraction mapping with Lipschitz coefficient γ on the complete pseudo-metric space $\langle \mathcal{Q}, d \rangle$, and Q^* is defined as in Theorem 1, then $\lim_{n \rightarrow \infty} d(\mathcal{T}^n Q, Q^*) = 0$ for any $Q \in \mathcal{Q}$.*

Theorems 1-3 guarantee that iteratively applying optimality operator $\mathcal{T}$ on any MOQ-value function will terminate with a function Q that is equivalent to Q^* under the measurement of pseudo-metric d . These Q s are as good as Q^* since they all have the same utilities for each ω , and will only differ when the utility corresponds to a recess in the frontier (see Figure 2(c) for an example, at the recess, either D or F is optimal).

Maintaining the envelope $\sup_{\omega'} \omega^\top Q(\cdot, \cdot, \omega')$ allows our method to quickly align one preference with optimal rewards and trajectories that may have been explored under other preferences, while scalarized updates that optimizes the scalar utility cannot use the information of $\max_a Q(s, a, \omega')$ to update the optimal solution aligned with a different ω . As illustrated in Figure 2 (c), assuming we have found two optimal solutions D and F in the CCS, misaligned with preferences ω_2 and ω_1 . The scalarized update cannot use the information of $\max_a Q(s, a, \omega_1)$ (corresponding to F) to update the optimal solution aligned with ω_2 or vice versa. It only searches along ω_1 direction leading to non-optimal L, even if solution D has been seen under ω_2 . Hence, the envelope updates can have better sample efficiency in theory, as is also seen from the empirical results.

Learning Algorithm. Using the above theorems, we provide a sample-efficient learning algorithm for multi-objective RL (Algorithm 1). Since our goal is to induce a single model that can adapt to the entire space of Ω , we use one parameterized function to represent $\mathcal{Q} \subseteq (\Omega \rightarrow \mathbb{R}^m)^{S \times \mathcal{A}}$. We achieve this by using a deep neural network with s, ω as input and $|\mathcal{A}| \times m$ Q -values as output. We then minimize the following loss function at each step k :²

$$L^A(\theta) = \mathbb{E}_{s, a, \omega} \left[\|\mathbf{y} - Q(s, a, \omega; \theta)\|_2^2 \right], \quad (6)$$

where $\mathbf{y} = \mathbb{E}_{s'}[r + \gamma \arg_Q \max_{a, \omega'} \omega^\top Q(s', a, \omega'; \theta_k)]$, which empirically can be estimated by sampling transition (s, a, s', r) from a replay buffer.

Optimizing L^A directly is challenging in practice because the optimal frontier contains a large number of discrete solutions, which makes the landscape of loss function considerably non-smooth. To address this, we use an auxiliary loss function L^B :

$$L^B(\theta) = \mathbb{E}_{s, a, \omega} [\|\omega^\top \mathbf{y} - \omega^\top Q(s, a, \omega; \theta)\|]. \quad (7)$$

²We use double Q learning with target Q networks following Mnih et al. [34]

Combined, our final loss function is $L(\theta) = (1 - \lambda) \cdot L^A(\theta) + \lambda \cdot L^B(\theta)$, where λ is a weight to trade off between losses L^A and L^B . We slowly increase the value of λ from 0 to 1, to shift our loss function from L^A to L^B . This method, known as *homotopy optimization* [12], is effective since for each update step, it uses the optimization result from the previous step as the initial guess. L^A first ensures the prediction of Q is close to any real expected total reward, although it may not be optimal. L^B provides an auxiliary pull along the direction with better utility.

The loss function above has an expectation over ω – this entails sampling random preferences in the algorithm. However, since the ω s are decoupled from the transitions, we can increase sample efficiency by using a scheme similar to Hindsight Experience Replay [11]. Furthermore, computing the optimality filter $\mathcal{H}$ over the entire Q is infeasible; instead we approximate this by applying $\mathcal{H}$ over a mini-batch of transitions before performing parameter updates. Further details on our model architectures and implementation details are available in the supplementary material (Section A.2.3).

Policy adaptation. Once we obtain a policy model $\Pi_{\mathcal{L}}(\omega)$ from the learning phase, the agent can adapt to any provided preference by simply feeding the ω into the network. While this is a straightforward scenario, we also consider a more challenging test where only scalar rewards are available and the agent has to uncover a hidden preference ω while adapting to the new task. For this case, we assume preferences are drawn from a truncated multivariable Gaussian distribution $\mathcal{D}_{\omega}^m(\mu_1, \dots, \mu_m; \sigma)$ on an $(m-1)$ -simplex, where nonnegative parameters $\mu_1, \dots, \mu_m$ are the means with $\mu_1 + \dots + \mu_m = 1$, and σ is a fixed standard deviation for all dimensions. Our goal is then to infer the parameters of this Gaussian distribution, for which we perform a combination of policy gradient (e.g., REINFORCE [35]) and stochastic search while keeping the policy model fixed. We determine the best preference parameters that maximize the expected return in the target task:

$$\arg \max_{\mu_1, \dots, \mu_m} \mathbb{E}_{\omega \sim \mathcal{D}_{\omega}^m} \left[\mathbb{E}_{\tau \sim (\mathcal{P}, \Pi_{\mathcal{L}}(\omega))} \left[\sum_{t=0}^{\infty} \gamma^t r_t(s_t, a_t) \right] \right]. \quad (8)$$

4 Experiments

Evaluation Metrics. Three metrics are to evaluate the empirical performance on test tasks:

a) *Coverage Ratio (CR)*. The first metric is *coverage ratio* (CR), which evaluates the agent’s ability to recover optimal solutions in the convex coverage set (CCS). If $\mathcal{F} \subseteq \mathbb{R}^m$ is the set of solutions found by the agent (via sampled trajectories), we define $\mathcal{F} \cap_{\epsilon} \text{CCS} := \{x \in \mathcal{F} \mid \exists y \in \text{CCS} \text{ s.t. } \|x - y\|_1 / \|y\|_1 \leq \epsilon\}$ as the intersection between these sets with a tolerance of ϵ . The CR is then defined as:

$$\text{CR}_{\mathcal{F}1}(\mathcal{F}) := 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}, \quad (9)$$

where the $\text{precision} = |\mathcal{F} \cap_{\epsilon} \text{CCS}| / |\mathcal{F}|$, indicating the fraction of optimal solutions among the retrieved solutions, and the $\text{recall} = |\mathcal{F} \cap_{\epsilon} \text{CCS}| / |\text{CCS}|$, indicating the fraction of optimal instances that have been retrieved over the total amount of optimal solutions (see Figure 3(a)).

Algorithm 1: Envelope MOQ-Learning

Input: a preference sampling distribution $\mathcal{D}_{\omega}$, path p_{λ} for the balance weight λ increasing from 0 to 1.
Initialize replay buffer $\mathcal{D}_{\tau}$, network Q_{θ} , and $\lambda = 0$.
for $\text{episode} = 1, \dots, M$ **do**
 Sample a linear preference $\omega \sim \mathcal{D}_{\omega}$.
 for $t = 0, \dots, N$ **do**
 Observe state s_t .
 Sample an action ϵ -greedily:

$$a_t = \begin{cases} \text{random action in } \mathcal{A}, & \text{w.p. } \epsilon; \\ \max_{a \in \mathcal{A}} \omega^{\top} Q(s_t, a, \omega; \theta), & \text{w.p. } 1 - \epsilon. \end{cases}$$

 Receive a vectorized reward $\mathbf{r}_t$ and observe s_{t+1} .
 Store transition $(s_t, a_t, \mathbf{r}_t, s_{t+1})$ in $\mathcal{D}_{\tau}$.
 if *update* **then**
 Sample N_{τ} transitions
 $(s_j, a_j, \mathbf{r}_j, s_{j+1}) \sim \mathcal{D}_{\tau}$.
 Sample N_{ω} preferences $W = \{\omega_i \sim \mathcal{D}_{\omega}\}$.
 Compute $y_{ij} = (\mathcal{T}Q)_{ij} =$

$$\begin{cases} \mathbf{r}_j, & \text{for terminal } s_{j+1}; \\ \mathbf{r}_j + \gamma \arg \max_{\substack{a \in \mathcal{A}, \\ \omega' \in W}} \omega_i^{\top} Q(s_{j+1}, a, \omega'; \theta), & \text{o.w.} \end{cases}$$

 for all $1 \leq i \leq N_{\omega}$ and $1 \leq j \leq N_{\tau}$.
 Update Q_{θ} by descending its stochastic gradient according to equations 6 and 7:

$$\nabla_{\theta} L(\theta) = (1 - \lambda) \cdot \nabla_{\theta} L^A(\theta) + \lambda \cdot \nabla_{\theta} L^B(\theta).$$

 Increase λ along the path p_{λ} .

b) *Adaptation Error (AE)*. Our second metric compares the retrieved control frontier with the optimal one, when an agent is provided with a specific preference ω during the adaptation phase:

$$AE(\mathcal{C}) := \mathbb{E}_{\omega \sim \mathcal{D}_\omega} [|\mathcal{C}(\omega) - \mathcal{C}_{\text{opt}}(\omega)| / \mathcal{C}_{\text{opt}}(\omega)], \quad (10)$$

which is the expected relative error between optimal control frontier $\mathcal{C}_{\text{opt}} : \Omega \rightarrow \mathbb{R}$ with $\omega \mapsto \max_{\hat{r} \in \text{CCS}} \omega^\top \hat{r}$ and the agent's control frontier $\mathcal{C}_{\pi_\omega} = \omega^\top \hat{r}_{\pi_\omega}$.

c) *Average Utility (UT)*. This measures the average utility obtained by the trained agent on randomly sampled preferences and is a useful proxy to AE when we don't have access to the optimal policy.

Domains. We evaluate on four different domains (complete details in supplementary material):

1. **Deep Sea Treasure (DST)** A classic MORL benchmark [14] in which an agent controls a submarine searching for treasures in a 10×11 -grid world while trading off time-cost and treasure-value. The grid world contains 10 treasures of different values. Their values increase as their distances from the starting point $s_0 = (0, 0)$ increase. We ensure the Pareto frontier of this environment to be convex.
2. **Fruit Tree Navigation (FTN)** A full binary tree of depth d with randomly assigned vectorial reward $r \in \mathbb{R}^6$ on the leaf nodes. These rewards encode the amounts of six different components of nutrition of the *fruits* on the tree: {Protein, Carbs, Fats, Vitamins, Minerals, Water}. For every leaf node, $\exists \omega$ for which its reward is optimal, thus all leaves lie on the CCS. The goal of our MORL agent is to find a path from the root to a leaf node that maximizes utility for a given preference, choosing between left or right subtrees at every non-terminal node.
3. **Task-Oriented Dialog Policy Learning (Dialog)** A modified task-oriented dialog system in the restaurant reservation domain based on PyDial [36]. We consider the task success rate and the dialog brevity (measured by number of turns) as two competing objectives of this domain.
4. **Multi-Objective SuperMario Game (SuperMario)** A multi-objective version of the popular video game Super Mario Bros. We modify the open-source environment from OpenAI gym [37] to provide vectorized rewards encoding five different objectives: *x-pos*: value corresponding to the difference in Mario's horizontal position between current and last time point, *time*: a small negative time penalty, *deaths*: a large negative penalty given each time Mario dies, *coin*: rewards for collecting coins, and *enemy*: rewards for eliminating an enemy.

Baselines. We compare our envelope MORL algorithm with classic and state-of-the-art baselines:

1. **MOFQI** [24]: Multi-objective fitted Q-iteration where the Q-approximator is a large linear model.
2. **CN+OLS** [13]: Conditional neural network with Optimistic Linear Support (OLS) method as the outer loop for selecting ω . This method is first proposed in [9] with multiple neural networks, and we employ an improved version using single conditional neural network [13].
3. **Scalarized** [13]: The state-of-the-art algorithm uses scalarized Q-update with double Q-learning, prioritized and hindsight experience replay, which is equivalent to CN+DER proposed in [13].

Main Results. Table 1 shows the performance comparison of different MORL algorithms in four domains. We elaborate training and test details for each domain in supplementary material. In DST and FTN we compare CR and AE as defined in section 4. In the task-oriented dialog policy learning task, we compare the average utility (Avg. UT) for 5,000 test dialogues with uniformly sampled user preferences on success and brevity. In the SuperMario game, the Avg. UT is over 500 test episodes

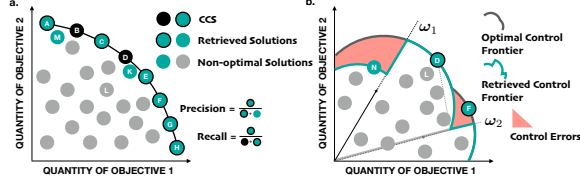


Figure 3: Illustration of evaluation metrics for MORL. (a.) Coverage ratio (CR) measures an agent's ability to find all the potential optimal solutions in the convex coverage set of Pareto frontier. Dots with black boundary are solutions in CCS, dots without black boundary are non-optimal returns, and dots in green are solutions retrieved by an MORL algorithm. CR is the F1 based on the precision and recall calculation. (b.) Adaptation error (AE) measures an agent's ability of policy adaptation to real-time specified preferences. The gray curve indicates the theoretical limit of the best cumulative utilities under all preference, and the green curve indicates the cumulative utilities of an MORL algorithm. AE is the average gap between these two curves over all preferences.

Method	DST		FTN ($d = 6$)		Dialog ²	SuperMario ²
	CR $\uparrow$	AE $\downarrow$	CR $\uparrow$	AE $\downarrow$	Avg. UT $\uparrow$	Avg. UT $\uparrow$
MOFQI	0.639 $\pm$ 0.421	139.6 $\pm$ 25.98	0.197 $\pm$ 0.000	0.176 $\pm$ 0.001	2.17 $\pm$ 0.21	–
CN+OLS	0.751 $\pm$ 0.163	34.63 $\pm$ 1.396	–	–	2.53 $\pm$ 0.22	–
Scalarized	0.989 $\pm$ 0.024	0.165 $\pm$ 0.096	0.914 $\pm$ 0.044	0.016 $\pm$ 0.005	2.38 $\pm$ 0.22	162.7 $\pm$ 77.66
Envelope (ours) ¹	0.994 $\pm$ 0.001	0.152 $\pm$ 0.006	0.987 $\pm$ 0.021	0.006 $\pm$ 0.001	2.65 $\pm$ 0.22	321.2 $\pm$ 146.9

Table 1: Comparison of different MORL algorithms in learning and adaptation phases across four experimental domains. $\uparrow$ indicates higher is better, and $\downarrow$ indicates lower is better for the scores. Each data point indicates the mean and standard deviation over 5 independent training and test runs. ¹Using the unpaired t-test, we obtain significance scores of $p < 0.05$ vs MOFQI on all domains, $p < 0.01$ vs CN+OLS on DST and $p < 0.05$ vs Scalarized on FTN, Dialog and SuperMario. ²Additional results are in the supplementary material C.4 and C.5.

with uniformly sampled preferences. The envelope algorithm steadily achieves the best performance in terms of both learning and adaptation among all the MORL methods in all four domains.

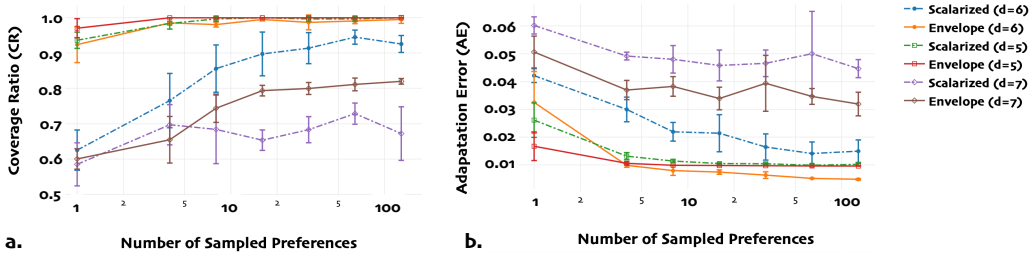


Figure 4: Coverage Ratio (CR) and Adaptation Error (AE) comparison of the scalarized algorithm [13] and our envelope deep MORL algorithm over 5000 episodes of FTN tasks of depths $d = 5, 6, 7$. Higher CR indicates better coverage of optimal policies, lower AE indicates better adaptation. The error bars are standard deviations of CR and AE estimated from 5 independent runs under each configuration.

Scalability. There are three aspects of the scalability of a MORL algorithm: the ability to deal with (1) large state space, (2) many objectives, and (3) large optimal policy set. Unlike other neural network-based methods, MOFQI cannot deal with the large state space, e.g., the video frames in SuperMario Game. The CN+OLS baseline requires solving all the intersection points of a set of hyper-planes thus is computationally intractable in domains with $m > 3$ objectives, such as FTN and SuperMario. We denote these entries as “–” in Table 1. Both scalarized and envelope methods can be applied to cases having large state space and reasonably many objectives. However, the size of optimal policy set may affect the performance of these algorithms. Figure 4 shows CR and AE results in three FTN environments with $d = 5$ (with 32 solutions), $d = 6$ (with 64 solutions), and $d = 7$ (with 128 solutions). We observe that both scalarized and envelope algorithms are close to optimal when $d = 5$ but both CR and AE values are worse for $d = 7$. However, the envelope version is more stable and outperforms the scalarized MORL algorithm in all three cases. These results point to the robustness and scalability of our algorithms.

Sample Efficiency. To compare sample efficiency during the learning phase, we train both our scalarized and envelope deep MORL on the FTN task with different depths for 5,000 episodes. We compute coverage ratio (CR) over 2,000 episodes and adaptation error (AE) over 5,000 episodes. Figure 4 shows plots for the metrics computed over a varying number of sampled preferences N_ω (more details can be found in the supplementary material). Each point on the curve is averaged over 5 experiments. We observe that the envelope MORL algorithm consistently has a better CR and AE scores than the scalarized version, with smaller variances. As N_ω increases, CR increases and AE decreases, which shows better use of historical interactions for both algorithms when N_ω is larger. And to achieve the same level AE the envelope algorithm requires smaller N_ω than the scalarized algorithm. This reinforces our theoretical analysis that the envelope MORL algorithm has better sample efficiency than the scalarized version.

Policy Adaptation. We show how the MORL agents respond to user preference during the adaptation phase in the dialog policy learning task, where the agent must trade off between the dialog success rate and the conversation brevity. Figure 5 shows the success rate (SR) curves as we vary the

	Protein	Carbs	Fats	Vitamins	Minerals	Water
v1	0.9639	0.	0.0361	0.	0.	0.
v2	0.	0.9067	0.	0.	0.	0.0933
v3	0.0539	0.	0.9461	0.	0.	0.
v4	0.1366	0.0459	0.	0.7503	0.0671	0.
v5	0.	0.0148	0.0291	0.0428	0.7503	0.1629
v6	0.	0.0505	0.	0.	0.	0.9495

Table 2: Inferred preferences of the envelope MOQ-learning algorithm on different FTN ($d = 6$) tasks (v1 to v6) after only 15 episodes interaction. The underlying preferences are all ones on the diagonal of the table and zeros for the off-diagonal.

	x-pos	time	life	coin	enemy
g1	0.5288	0.1770	0.1500	0.0470	0.0972
g2	0.1985	0.2237	0.2485	0.1422	0.1868
g3	0.2196	0.1296	0.3541	0.1792	0.1175
g4	0.0211	0.2404	0.0211	0.6960	0.0211
g5	0.0715	0.1038	0.2069	0.3922	0.2253

Table 3: Inferred preferences of the envelope multi-objective A3C algorithm in different Mario Game variants (g1 to g5) with 100 episodes. The underlying preferences are all ones on the diagonal of the table and zeros for the off-diagonal.

weight of the preference on task completion success. The success rates of both MORL algorithms increase as the user’s weight on success increases, while those of the single-objective algorithms do not change. This shows that our envelope MORL agent can adapt gracefully to the user’s preference. Furthermore, our envelope deep MORL algorithm outperforms other algorithms whenever success is relatively more important to the user (weight > 0.5).

Revealing underlying preferences. Finally, we test the ability of our agent to infer and adapt to unknown preferences on FTN and SuperMario. During the learning phase, the agent does not know the underlying preference, and hence learns a multi-objective policy. During the adaptation phase, our agent performs recovery underlying preferences (as described in Section 3) to uncover the underlying preference that maximizes utility. Table 2 shows the learned preferences for 6 different FTN tasks (v1 to v6) with unknown one-hot preferences $[1, 0, 0, 0, 0, 0]$ to $[0, 0, 0, 0, 0, 1]$, respectively, meaning the agent should only care about one elementary nutrition. These were learned in a few-shot adaption setting, using just 15 episodes. For the SuperMario Game, we implement an A3C [38] variant of our envelope MORL agent (see supplementary material for details). Table 3 shows the learned preferences for 5 different tasks (g1 to g5) with unknown one-hot preferences using just 100 episodes.

We observe that the learned preferences are concentrated on the diagonal, indicating good alignment with the actual underlying preferences. For example, in the SuperMario game variant g4, the envelope MORL agent finds the preference with the highest weight (0.6960) on the coin objective can best describe the goal of g4, which is to collect as many coins as possible. We also tested policy adaptation on the original Mario game using game scores for the scalar rewards. We find that the agent learns preference weights of 0.37 for x-pos and 0.23 for time, which seems consistent with a common strategy that humans employ – simply move Mario towards the flag as quickly as possible.

5 Conclusion

We have introduced a new algorithm for multi-objective reinforcement learning (MORL) with linear preferences, with the goal of enabling few-shot adaptation of autonomous agents to new scenarios. Specifically, we propose a multi-objective version of the Bellman optimality operator, and utilize it to learn a single parametric representation for all optimal policies over the space of preferences. We provide convergence proofs for our multi-objective algorithm and also demonstrate how to use our model to adapt and elicit an unknown preference on a new task. Our experiments across four different domains demonstrate that our algorithms exhibit effective generalization and policy adaptation.

Acknowledgements

The authors would like to thank Yexiang Xue at Purdue University, Carla Gomes at Cornell University for helpful discussions on multi-objective optimization, Lu Chen, Kai Yu at Shanghai Jiao Tong University for discussing dialogue applications, Haoran Cai at MIT for helping running a part of synthetic experiments, and anonymous reviewers for constructive suggestions.



Figure 5: The success-weight curves of task-oriented dialog. Each data point is a moving average of closest around 500 dialogues in the interval of around ± 0.05 weight of success. The light shadow indicates the standard deviations of 5 independent runs under each configuration.

References

- [1] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in ai safety. *arXiv preprint arXiv:1606.06565*, 2016.
- [2] Diederik M. Roijers, Peter Vamplew, Shimon Whiteson, and Richard Dazeley. A survey of multi-objective sequential decision-making. *J. Artif. Intell. Res.*, 48:67–113, 2013.
- [3] Il Yong Kim and OL De Weck. Adaptive weighted sum method for multiobjective optimization: a new method for pareto front generation. *Structural and multidisciplinary optimization*, 31(2):105–116, 2006.
- [4] Abdullah Konak, David W Coit, and Alice E Smith. Multi-objective optimization using genetic algorithms: A tutorial. *Reliability Engineering & System Safety*, 91(9):992–1007, 2006.
- [5] Hirotaka Nakayama, Yeboon Yun, and Min Yoon. *Sequential approximate multiobjective optimization using computational intelligence*. Springer Science & Business Media, 2009.
- [6] JiGuan G Lin. On min-norm and min-max methods of multi-objective optimization. *Mathematical programming*, 103(1):1–33, 2005.
- [7] Sriraam Natarajan and Prasad Tadepalli. Dynamic preferences in multi-criteria reinforcement learning. In Luc De Raedt and Stefan Wrobel, editors, *Machine Learning, Proceedings of the Twenty-Second International Conference (ICML 2005), Bonn, Germany, August 7-11, 2005*, volume 119 of *ACM International Conference Proceeding Series*, pages 601–608. ACM, 2005.
- [8] Leon Barrett and Sridhar Narayanan. Learning all optimal policies with multiple criteria. In William W. Cohen, Andrew McCallum, and Sam T. Roweis, editors, *Machine Learning, Proceedings of the Twenty-Fifth International Conference (ICML 2008), Helsinki, Finland, June 5-9, 2008*, volume 307 of *ACM International Conference Proceeding Series*, pages 41–47. ACM, 2008.
- [9] Hossam Mossalam, Yannis M. Assael, Diederik M. Roijers, and Shimon Whiteson. Multi-objective deep reinforcement learning. *CoRR*, abs/1610.02707, 2016.
- [10] Richard Ernest Bellman. *Dynamic Programming*. Princeton University Press, Princeton, NJ, USA, 1957.
- [11] Marcin Andrychowicz, Dwight Crow, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, 4-9 December 2017, Long Beach, CA, USA*, pages 5055–5065, 2017.
- [12] Layne T Watson and Raphael T Haftka. Modern homotopy methods in optimization. *Computer Methods in Applied Mechanics and Engineering*, 74(3):289–305, 1989.
- [13] Axel Abels, Diederik M. Roijers, Tom Lenaerts, Ann Nowé, and Denis Steckelmacher. Dynamic weights in multi-objective deep reinforcement learning. In *Proceedings of the 36th International Conference on Machine Learning*, page TBA, 2019.
- [14] Peter Vamplew, Richard Dazeley, Adam Berry, Rustam Issabekov, and Evan Dekker. Empirical evaluation methods for multiobjective reinforcement learning algorithms. *Machine Learning*, 84(1-2):51–80, 2011.
- [15] Chunming Liu, Xin Xu, and Dewen Hu. Multiobjective reinforcement learning: A comprehensive overview. *IEEE Trans. Systems, Man, and Cybernetics: Systems*, 45(3):385–398, 2015.
- [16] Shie Mannor and Nahum Shimkin. The steering approach for multi-criteria reinforcement learning. In Thomas G. Dietterich, Suzanna Becker, and Zoubin Ghahramani, editors, *Advances in Neural Information Processing Systems 14 [Neural Information Processing Systems: Natural and Synthetic, NIPS 2001, December 3-8, 2001, Vancouver, British Columbia, Canada]*, pages 1563–1570. MIT Press, 2001.
- [17] Gerald Tesauro, Rajarshi Das, Hoi Chan, Jeffrey O. Kephart, David Levine, Freeman L. Rawson III, and Charles Lefurgy. Managing power consumption and performance of computing systems using reinforcement learning. In John C. Platt, Daphne Koller, Yoram Singer, and Sam T. Roweis, editors, *Advances in Neural Information Processing Systems 20, Proceedings of the Twenty-First Annual Conference on Neural Information Processing Systems, Vancouver, British Columbia, Canada, December 3-6, 2007*, pages 1497–1504. Curran Associates, Inc., 2007.
- [18] Kristof Van Moffaert, Madalina M. Drugan, and Ann Nowé. Scalarized multi-objective reinforcement learning: Novel design techniques. In *Proceedings of the 2013 IEEE Symposium on Adaptive Dynamic Programming and Reinforcement Learning, ADPRL 2013, IEEE Symposium Series on Computational Intelligence (SSCI), 16-19 April 2013, Singapore*, pages 191–199. IEEE, 2013.

- [19] Matteo Pirotta, Simone Parisi, and Marcello Restelli. Multi-objective reinforcement learning with continuous pareto frontier approximation. In Blai Bonet and Sven Koenig, editors, *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence, January 25-30, 2015, Austin, Texas, USA.*, pages 2928–2934. AAAI Press, 2015.
- [20] Simone Parisi, Matteo Pirotta, and Jan Peters. Manifold-based multi-objective policy search with sample reuse. *Neurocomputing*, 263:3–14, 2017.
- [21] Kazuyuki Hiraoka, Manabu Yoshida, and Taketoshi Mishima. Parallel reinforcement learning for weighted multi-criteria model with adaptive margin. In Masumi Ishikawa, Kenji Doya, Hiroyuki Miyamoto, and Takeshi Yamakawa, editors, *Neural Information Processing, 14th International Conference, ICONIP 2007, Kitakyushu, Japan, November 13-16, 2007, Revised Selected Papers, Part I*, volume 4984 of *Lecture Notes in Computer Science*, pages 487–496. Springer, 2007.
- [22] Hitoshi Iima and Yasuaki Kuroe. Multi-objective reinforcement learning for acquiring all pareto optimal policies simultaneously - method of determining scalarization weights. In *2014 IEEE International Conference on Systems, Man, and Cybernetics, SMC 2014, San Diego, CA, USA, October 5-8, 2014*, pages 876–881. IEEE, 2014.
- [23] Andrea Castelletti, Francesca Pianosi, and Marcello Restelli. Multi-objective fitted q-iteration: Pareto frontier approximation in one single run. In *Proceedings of the IEEE International Conference on Networking, Sensing and Control, ICNSC 2011, Delft, The Netherlands, 11-13 April 2011*, pages 260–265. IEEE, 2011.
- [24] Andrea Castelletti, Francesca Pianosi, and Marcello Restelli. Tree-based fitted q-iteration for multi-objective markov decision problems. In *The 2012 International Joint Conference on Neural Networks (IJCNN), Brisbane, Australia, June 10-15, 2012*, pages 1–8. IEEE, 2012.
- [25] Wolfram Conen and Tuomas Sandholm. Preference elicitation in combinatorial auctions. In *Proceedings of the 3rd ACM conference on Electronic Commerce*, pages 256–259. ACM, 2001.
- [26] Craig Boutilier. A POMDP formulation of preference elicitation problems. In *Proceedings of the Eighteenth National Conference on Artificial Intelligence and Fourteenth Conference on Innovative Applications of Artificial Intelligence, July 28 - August 1, 2002, Edmonton, Alberta, Canada.*, pages 239–246, 2002.
- [27] Li Chen and Pearl Pu. Survey of preference elicitation methods. Technical report, 2004.
- [28] Andrew Y. Ng and Stuart J. Russell. Algorithms for inverse reinforcement learning. In Pat Langley, editor, *Proceedings of the Seventeenth International Conference on Machine Learning (ICML 2000), Stanford University, Stanford, CA, USA, June 29 - July 2, 2000*, pages 663–670. Morgan Kaufmann, 2000.
- [29] Pieter Abbeel and Andrew Y. Ng. Apprenticeship learning via inverse reinforcement learning. In Carla E. Brodley, editor, *Machine Learning, Proceedings of the Twenty-first International Conference (ICML 2004), Banff, Alberta, Canada, July 4-8, 2004*, volume 69 of *ACM International Conference Proceeding Series*. ACM, 2004.
- [30] Jonathan Ho and Stefano Ermon. Generative adversarial imitation learning. In Daniel D. Lee, Masashi Sugiyama, Ulrike von Luxburg, Isabelle Guyon, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 29: Annual Conference on Neural Information Processing Systems 2016, December 5-10, 2016, Barcelona, Spain*, pages 4565–4573, 2016.
- [31] Urszula Chajewska, Daphne Koller, and Dirk Ormoneit. Learning an agent’s utility function by observing behavior. In *Proceedings of the Eighteenth International Conference on Machine Learning (ICML 2001), Williams College, Williamstown, MA, USA, June 28 - July 1, 2001*, pages 35–42, 2001.
- [32] Christopher J. C. H. Watkins and Peter Dayan. Technical note q-learning. *Machine Learning*, 8:279–292, 1992.
- [33] Arch W Naylor and George R Sell. *Linear operator theory in engineering and science*. Springer Science & Business Media, 2000.
- [34] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin A. Riedmiller, Andreas Fiedjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharmashan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [35] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.

- [36] Stefan Ultes, Lina Maria Rojas-Barahona, Pei-Hao Su, David Vandyke, Dongho Kim, Iñigo Casanueva, Pawel Budzianowski, Nikola Mrksic, Tsung-Hsien Wen, Milica Gasic, and Steve J. Young. Pydial: A multi-domain statistical dialogue system toolkit. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics, ACL 2017, Vancouver, Canada, July 30 - August 4, System Demonstrations*, pages 73–78, 2017.
- [37] Christian Kauten. Super Mario Bros for OpenAI Gym. <https://github.com/Kautenja/gym-super-mario-bros>, 2018.
- [38] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy P. Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19-24, 2016*, pages 1928–1937, 2016.
- [39] Mohamed A Khamsi and William A Kirk. *An introduction to metric spaces and fixed point theory*, volume 53. John Wiley & Sons, 2011.
- [40] Dimitri P. Bertsekas. Regular policies in abstract dynamic programming. *SIAM Journal on Optimization*, 27(3):1694–1727, 2017.
- [41] Dimitri P Bertsekas. *Abstract dynamic programming*. Athena Scientific Belmont, MA, 2018.
- [42] Marc G. Bellemare, Will Dabney, and Rémi Munos. A distributional perspective on reinforcement learning. In *Proceedings of the 34th International Conference on Machine Learning, ICML 2017, Sydney, NSW, Australia, 6-11 August 2017*, pages 449–458, 2017.
- [43] Jost Schatzmann and Steve J. Young. The hidden agenda user simulation model. *IEEE Trans. Audio, Speech & Language Processing*, 17(4):733–747, 2009.
- [44] Stefan Ultes, Pawel Budzianowski, Iñigo Casanueva, Nikola Mrksic, Lina Maria Rojas-Barahona, Pei-Hao Su, Tsung-Hsien Wen, Milica Gasic, and Steve J. Young. Reward-balancing for statistical spoken dialogue systems using multi-objective reinforcement learning. In *Proceedings of the 18th Annual SIGdial Meeting on Discourse and Dialogue, Saarbrücken, Germany, August 15-17, 2017*, pages 65–70, 2017.
- [45] Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *CoRR*, abs/1509.02971, 2015.
- [46] Shixiang Gu, Timothy P. Lillicrap, Ilya Sutskever, and Sergey Levine. Continuous deep q-learning with model-based acceleration. In *Proceedings of the 33rd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19-24, 2016*, pages 2829–2838, 2016.
- [47] Luke Metz, Julian Ibarz, Navdeep Jaitly, and James Davidson. Discrete sequential prediction of continuous actions for deep RL. *CoRR*, abs/1705.05035, 2017.
- [48] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In Dale Schuurmans and Michael P. Wellman, editors, *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence, February 12-17, 2016, Phoenix, Arizona, USA.*, pages 2094–2100. AAAI Press, 2016.
- [49] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. *CoRR (Published at ICLR 2016)*, abs/1511.05952, 2015.
- [50] L.J.P van der Maaten and G.E. Hinton. Visualizing high-dimensional data using t-sne. *Journal of Machine Learning Research*, Nov 2008.

Supplementary Material for Generalized Algorithm for Multi-Objective RL and Policy Adaptation

A Theoretical Framework for Value-Based MORL Algorithms

In this section, we introduce a theoretical framework for analyzing and designing the value-based multi-objective reinforcement learning algorithms. This framework is based on the well-known Banach's Fixed-Point Theorem, which guarantees the existence and uniqueness of fixed-point of a *contraction* on a *complete metric space*. Therefore, generalizing this theorem a bit, we can imagine all value functions of reinforcement learning are in some metric space, and finding the optimal value or policy is to find the fixed point of a certain contraction on that space. We first recall the following concepts.

A.1 General Framework for Value-Based Reinforcement Learning

Definition 1 (Contraction). *Let (X, d) be a metric space. We say that $\mathcal{T}$ is a contraction, if there is a real number $\gamma \in [0, 1)$ such that*

$$d(\mathcal{T}(x), \mathcal{T}(x')) \leq \gamma d(x, x') \quad (11)$$

for all points $x, x' \in X$, where γ is called a Lipschitz coefficient for the contraction $\mathcal{T}$.

Theorem 4 (Banach's Fixed-Point Theorem). *Let (X, d) be a complete metric space and let $\mathcal{T} : X \rightarrow X$ be a contraction. Then there exists a **unique** fixed point $x^* \in X$ such that $\mathcal{T}(x^*) = x^*$. Moreover, if x is any point in X and $\mathcal{T}^n(x)$ is inductively defined by $\mathcal{T}^n(x) = \mathcal{T}(\mathcal{T}^{n-1}(x))$, then we have $\mathcal{T}^n(x) \rightarrow x^*$ as $n \rightarrow \infty$.*

The above introduced Banach fixed-point theorem is well-known. Readers may refer to the book [39] for more details. Practically, this provides us with an iterative method for converging to any desired solution in the large solution space, by repeatedly applying a properly designed contraction. For example, the foundation for standard value-based single-objective reinforcement learning is the use of Bellman's optimality equation [10]:

$$Q^*(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim \mathcal{P}(\cdot|s, a)} \sup_{a' \in \mathcal{A}} Q^*(s', a'), \quad (12)$$

where $\gamma \in [0, 1)$ is the discount factor and the optimal Q-value function $Q^*(s, a)$ is the desired solution in the space $\mathcal{Q} \subseteq \mathbb{R}^{S \times A}$ consisting of all the bounded functions with ℓ_∞ -distance metric

$$d(Q, Q') := \sup_{s \in S, a \in A} |Q(s, a) - Q'(s, a)|. \quad (13)$$

Since the all the functions in this space is bounded, it follows that with this ℓ_∞ -distance metric, the space $(\mathcal{Q}, d)$ is complete. Besides, according to the equation (12), we can design an *Bellman optimality operator* $\mathcal{T}$ such that

$$(\mathcal{T}Q)(s, a) := r(s, a) + \gamma \mathbb{E}_{s' \sim \mathcal{P}(\cdot|s, a)} \sup_{a' \in \mathcal{A}} Q(s', a'), \quad (14)$$

which can be shown as a contraction on $(\mathcal{Q}, d)$. Many popular value-based reinforcement algorithms, such as *deep Q-learning* [34], can be seen as asynchronous iteration methods with approximately applied contraction.

We can verify that the Bellman optimality operator $\mathcal{T}$ indeed is a contraction on $(\mathcal{Q}, d)$, and the optimal value function Q^* is a fixed point in $(\mathcal{Q}, d)$. Therefore, we can find the unique optimal Q-value function by applying the optimality operator iteratively many times on any initial Q-value function. Similarly, we can also define *Bellman evaluation operator* $\mathcal{T}_\pi$ using the Bellman expectation equation $(\mathcal{T}_\pi Q)(s, a) := r(s, a) + \gamma \mathbb{E}_{\tau \sim (\mathcal{P}, \pi)} Q(s', a')$, which is also a contraction.

Knowing that the optimality operator is a contraction is important. In practice, we use a minibatch to update previous Q-value function approximated by neural networks, not updating all states and actions. Thus the updated Q-value function is not a strict $\mathcal{T}Q$, but only close to $\mathcal{T}Q$ on some state and action pairs. We can still provide a theoretical guarantee that a minibatch iterative algorithm can still converge to a promising result, under certain extra assumptions.

Definition 2 (Minibatch Iteration). *Consider the Q-value function Q as a composition of $\{Q_{S,A}\}_{S \subseteq S, A \subseteq A}$ such that in each iteration,*

$$Q_{S,A}^{k+1}(s, a) := \begin{cases} \mathcal{T}Q_{S,A}^k(s, a), & \text{if } s \in S \text{ and } a \in A; \\ Q_{S,A}^k(s, a), & \text{otherwise.} \end{cases}$$

Theorem 5 (Minibatch Convergence Theorem). *Suppose each restricted Q-value function $Q_{S,A}$ can be update an arbitrary number of times, and there is a nest sequence of nonempty sets $\{Q^k\}_{k \in \mathbb{Z}}$ with $Q^{k+1} \subseteq Q^k \subseteq \mathcal{Q}$, $k = 0, 1, \dots$ such that if $\{Q^k\}_{k \in \mathbb{N}}$ is any sequence with $Q^k \in \mathcal{Q}^k$ for all $k \geq 0$, then $\{Q^k\}$ converges pointwisely to Q^* . Assume further the following:*

1. *Convergence Condition: We have*

$$\forall Q \in \mathcal{Q}^k, \mathcal{T}Q \in \mathcal{Q}^{k+1}; \quad (15)$$

2. *Box Condition: For all k , $\mathcal{Q}^k$ is a Cartesian product of the form*

$$\mathcal{Q}^k = \times_{s \in \mathcal{S}, a \in \mathcal{A}} \mathcal{Q}_{\{s\}, \{a\}}^k, \quad (16)$$

where $\mathcal{Q}_{S,A}^k$ is a set of bounded real-valued functions on states S and actions A .

Then for every $Q^0 \in \mathcal{Q}^0$ the sequence $\{Q^k\}$ generated by the minibatch iteration algorithm converges to Q^ [40].*

Proof. Showing the convergence of the algorithm is equivalent to showing that the iterations of elements from $\mathcal{Q}^k$ will get in to $\mathcal{Q}^{k+1}$ eventually, i.e., for each $k \geq 0$, there is a time t_k such that $Q^t \in \mathcal{Q}^k$ for all $t \geq t_k$. We can prove it by mathematical induction.

When $k = 0$, the statement is true since $Q^0 \in \mathcal{Q}^0$. Assuming the statement is true for a given k , we will show there is a time t_{k+1} with the required properties. For each $s \in \mathcal{S}, a \in \mathcal{A}$, let set $L_{s,a} = \{t : s^t = s, a^t = a\}$ record the time a minibatch update happens on the state s and action a . Let $t(s, a)$ be the first element in $L_{s,a}$ such that $t(s, a) \geq t_k$. Then by the convergence condition, we have $\mathcal{T}Q^{t(s,a)} \in \mathcal{Q}^{k+1}$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}$. In the view of the box condition, it implies $Q^{t+1}(s, a) \in \mathcal{Q}_{s,a}^{k+1}$ for all $t \geq t(s, a)$ for any $s \in \mathcal{S}$ and $a \in \mathcal{A}$. Therefore, let $t_{k+1} = \max_{s,a} t(s, a) + 1$, using the box condition, we have $Q^t \in \mathcal{Q}^{k+1}$ for all $t \leq t_{k+1}$. By mathematical induction, the statement holds, and $\mathcal{Q}^k$ will shrink in to $\mathcal{Q}^{k+1}$ eventually. Hence, we have proved $\{Q^t\}$ converges to Q^* finally. $\square$

The above theorem indicates that minibatch update with experience replay will not affect the convergence of iteratively applying the optimality operator $\mathcal{T}$. This gives us the flexibility to design value-based algorithm's updating scheme. Besides, even though we use deep Q-network to approximate the Q-value function and update it by using gradient descent, this will not impair the magical function of optimality operator too much. This is because if after n -round neural network updates, we always have $\sup_{s \in \mathcal{S}, a \in \mathcal{A}} [Q^{t+n} - \mathcal{T}Q^t] \leq \epsilon$, where n is an arbitrary bounded integer, by applying the triangle inequality we conclude that the final error $d(Q_{final}, Q^*) \leq \epsilon/(1 - \gamma)$ is bounded.

We summarize a general theoretical framework for value-based RL algorithm design, which consists of five elements:

- **Value Space $\mathcal{X}$:** A common choice of $\mathcal{X}$ is the space of value functions in $\mathbb{R}^{\mathcal{S}}$ or the space of Q-value functions in $\mathbb{R}^{\mathcal{S} \times \mathcal{A}}$. There are many other choices such as the space of ordered pair of (V, Q) (see 2.6.2 in book [41] for example), or the space of vectorized value functions as we show in the following sections.
- **Value Metric d :** It defines the "distance" between two points in the value space. Besides the basic four requirements, the metric d should ensure a complete metric space $(\mathcal{X}, d)$ to validate the Banach's fixed-point theorem. A compatible selection of value metric will make the convergence analysis easier.
- **Evaluation Operator $\mathcal{T}_\pi$:** We have constructed a recursive expression of a certain value point in the value space associated with some policy, e.g., the Bellman expectation equation, to depict the value of a policy v_π as a fixed point we desire. Carefully verify that the contraction property holds for $\mathcal{T}_\pi$.
- **Optimality Operator $\mathcal{T}$:** A recursive expression of the optimal point in the value space, e.g., the Bellman optimality equation. Note that when the metric d is the supremum of the absolute value of the difference, and $\mathcal{T}_\pi$ is a contraction, we can prove the contraction property of $\mathcal{T}$ is always automatically satisfied.
- **Updating Scheme:** To make a reinforcement learning algorithm practical and scalable, we need to consider many factors in terms of updating scheme. For example: How do we approximate the value and policies? If it is an online algorithm, how do we trade off exploration and exploitation? All these details will significantly influence the performance of our algorithm on real-world tasks.

In summary, there are five essential components for analyzing and designing general value-base reinforcement learning algorithms: (1) **value space**, (2) **value metric**, (3) **evaluation operator**, (4) **optimality operator**, and (5) **updating scheme**. In fact, there is some work [42] developing distributional reinforcement learning in a way similar to this framework. We will discuss how to design these five elements of our framework to develop envelope multi-objective reinforcement learning algorithms in the next section.

A.2 MORL with envelope updates

The deep MORL algorithm with scalarized update is capable of solving unknown linear preference scenarios of multi-objective reinforcement learning. However, there are several limitations of this algorithm restrict its applicability and performance in practice. Aiming at solving problems stated in Section 2, we design a new algorithm called envelope deep MORL algorithm. Following the value-based theoretical framework introduced in Section A.1, our key idea to upgrade the scalarized algorithm is to consider a different value space, where every Q-value function is a mapping to multi-objective solutions, not utilities, and therefore maintains the necessary information for prediction in the adaptation phase. Furthermore, we generalize the optimality filter $\mathcal{H}$ to use that information to boost up the alignment in the learning phase.

We consider a new value space $\mathcal{Q} \subseteq (\Omega \rightarrow \mathbb{R}^m)^{\mathcal{S} \times \mathcal{A}}$, containing all bounded functions $Q(s, a, \omega)$ returning the estimates of preferred expected total rewards under preference ω , which are m -dimensional vectors. Besides, we employ a value metric d defined by

$$d(Q, Q') := \sup_{\substack{s \in \mathcal{S}, a \in \mathcal{A} \\ \omega \in \Omega}} |\omega^\top (Q(s, a, \omega) - Q'(s, a, \omega))|. \quad (17)$$

Notice that this metric is a pseudo-metric, since the identity of indiscernibles does not hold for it. It is easy to show that metric space $(\mathcal{Q}, d)$ is complete.

We refer the Q-value functions in this $\mathcal{Q}$ *multi-objective value functions*. Similar to the scalarized one, we design an evaluation operator and an optimality operator for this envelope version algorithm. As for the updating scheme, we use hindsight experience replay and a homotopy optimization trick.

A.2.1 Multi-Objective Bellman Optimality Operator

In this section, we give the evaluation operator and the optimality operator in the new envelope version value space $(\mathcal{Q}, d)$ as stated above. The evaluation operator now is even simpler than that of the scalarized version. Give a policy π , the evaluation operator is defined by

$$(\mathcal{T}_\pi Q)(s, a, \omega) := r(s, a) + \gamma \mathbb{E}_{s' \sim \mathcal{P}(\cdot | s, a)} Q(s', a', \omega). \quad (18)$$

Since now the multi-objective Q-value function is also in a vector form, this evaluation operator is almost the same as that of the single-objective reinforcement learning. It can be easily verified as a contraction.

As for the envelope version of optimality operator, we employ a stronger optimality filter $\mathcal{H}$, defined by $(\mathcal{H}Q)(s, \omega) := \arg_Q \sup_{a' \in \mathcal{A}, \omega' \in \Omega} \omega^\top Q(s, a', \omega')$, where the $\arg_Q$ takes the multi-objective value corresponding to the supremum, i.e., $Q(s, a'', \omega'')$ such that $(a'', \omega'') \in \arg \sup_{a' \in \mathcal{A}, \omega' \in \Omega} \omega^\top Q(s, a', \omega')$. The return of $\arg_Q$ depends on scalarization weights ω , and we use $\arg_Q$ for simplicity of notation. This filter is solving the convex envelope of the current Pareto frontier, therefore we name this algorithm as “envelope” version. We can write the optimality operator $\mathcal{T}$ in terms of the optimal filter:

$$(\mathcal{T}Q)(s, a, \omega) := r(s, a) + \gamma \mathbb{E}_{s' \sim \mathcal{P}(\cdot | s, a)} (\mathcal{H}Q)(s', \omega). \quad (19)$$

Theorem 1 (Fixed Point of Envelope Optimality Operator for MORL). *Use above definitions in the envelope version value space. Let $Q^* \in \mathcal{Q}$ be the preferred optimal value function in the value space, such that*

$$Q^*(s, a, \omega) = \arg_Q \sup_{\pi \in \Pi} \omega^\top \mathbb{E}_{\tau \sim (\mathcal{P}, \pi)} |_{s_0=s, a_0=a} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right], \quad (20)$$

where the $\arg_Q$ takes the multi-objective value corresponding to the supremum, then $Q^* = \mathcal{T}Q^*$ holds.

Proof. First, we observe that $d(Q^*, \mathcal{T}Q^*) = \sup_{\substack{s \in \mathcal{S}, a \in \mathcal{A} \\ \omega \in \Omega}} |\omega^\top (Q^*(s, a, \omega) - \mathcal{T}Q^*(s, a, \omega))| = 0 \Leftrightarrow \omega^\top \mathcal{T}Q^*(s, a, \omega) = \omega^\top Q^*(s, a, \omega)$ for all s, a, ω . Then, by substituting the definition of Q^* into $\mathcal{T}Q^*$,

$$\begin{aligned} \omega^\top \mathcal{T}Q^*(s, a, \omega) &= \omega^\top r(s, a) + \gamma \cdot \omega^\top \mathbb{E}_{s' \sim \mathcal{P}(\cdot | s, a)} (\mathcal{H}Q^*)(s', \omega) \\ (\text{def. of } \mathcal{H}) &= \omega^\top r(s, a) + \gamma \cdot \omega^\top \mathbb{E}_{s' \sim \mathcal{P}(\cdot | s, a)} \arg_Q \sup_{a' \in \mathcal{A}, \omega' \in \Omega} \omega^\top Q^*(s', a', \omega') \\ (\text{linearity of exp. \& cancel } \omega^\top \text{ and } \arg_Q) &= \omega^\top r(s, a) + \gamma \cdot \mathbb{E}_{s' \sim \mathcal{P}(\cdot | s, a)} \sup_{a' \in \mathcal{A}, \omega' \in \Omega} \omega^\top Q^*(s', a', \omega') \\ (\text{insert eq. (20), def. of } Q^*) &= \omega^\top r(s, a) + \gamma \cdot \mathbb{E}_{s' \sim \mathcal{P}(\cdot | s, a)} \sup_{a' \in \mathcal{A}, \omega' \in \Omega} \omega^\top \left\{ \arg_Q \sup_{\pi \in \Pi} \omega'^\top \mathbb{E}_{\tau \sim (\mathcal{P}, \pi)} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right] \right\} \\ (\text{use def. of } \arg_Q, \text{ explained below}) &= \omega^\top r(s, a) + \gamma \cdot \mathbb{E}_{s' \sim \mathcal{P}(\cdot | s, a)} \sup_{a' \in \mathcal{A}} \omega^\top \left\{ \arg_Q \sup_{\pi \in \Pi} \omega'^\top \mathbb{E}_{\tau \sim (\mathcal{P}, \pi)} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right] \right\} \\ (\text{rearrange expectation and sup}) &= \omega^\top r(s, a) + \gamma \cdot \omega^\top \arg_Q \sup_{\pi \in \Pi} \omega^\top \mathbb{E}_{\tau \sim (\mathcal{P}, \pi)} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right] \\ (\text{merge 1st term to sum \& use def. of } Q^* \text{ again}) &= \omega^\top \left\{ \arg_Q \sup_{\pi \in \Pi} \omega^\top \mathbb{E}_{\tau \sim (\mathcal{P}, \pi)} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right] \right\} = \omega^\top Q^*(s, a, \omega) \end{aligned}$$

The fourth equation is due to a sandwich inequality, $\omega^\top \arg_Q \sup_{\pi \in \Pi} \omega^\top Q^\pi \leq \sup_{\omega' \in \Omega} \omega^\top \arg_Q \sup_{\pi \in \Pi} \omega'^\top Q^\pi = \omega^\top \arg_Q \sup_{\pi \in \Pi} \omega_*^\top Q^\pi = \omega^\top Q^{\pi_{\omega_*}'} \leq \omega^\top \arg_Q \sup_{\pi \in \Pi} \omega^\top Q^\pi$, where ω_* and π_{ω_*}' are preference and policy corresponding to the supremums. According to the observation stated at the beginning, $d(Q^*, \mathcal{T}Q^*) = 0$. The preferred optimal value function is a fixed point of the proposed envelope version optimality operator. $\square$

Theorem 1 tells us the preferred optimal value function is one of the fixed-points of envelope optimality operator $\mathcal{T}$ in the value space. And we still need to show that this $\mathcal{T}$ is a contraction.

Theorem 2 (Envelope Optimal Operator is a Contraction). *Let Q, Q' be any two multi-objective Q -value functions in the envelope value space $\mathcal{Q}$ as defined above, the Lipschitz condition $d(\mathcal{T}Q, \mathcal{T}Q') \leq \gamma d(Q, Q')$ holds, where $\gamma \in [0, 1)$ is the discount factor of the underlying MOMDP $\mathcal{M}$ (see Section 2).*

Proof. Without the loss of generality, we assume $\sup_{a \in \mathcal{A}, \omega' \in \Omega} \omega^\top Q(s, a, \omega') \geq \sup_{a \in \mathcal{A}, \omega' \in \Omega} \omega^\top Q'(s, a, \omega')$ for some state s and ω of interest. Expand the expression of $d(\mathcal{T}Q, \mathcal{T}Q')$ we have

$$\begin{aligned} d(\mathcal{T}Q, \mathcal{T}Q')(s, a) &= \sup_{\substack{s \in \mathcal{S}, a \in \mathcal{A} \\ \omega \in \Omega}} |\omega^\top ((\mathcal{T}Q)(s, a) - (\mathcal{T}Q')(s, a))| \\ &= \sup_{\substack{s \in \mathcal{S}, a \in \mathcal{A} \\ \omega \in \Omega}} |\gamma \cdot \omega^\top \mathbb{E}_{s' \sim P(\cdot | s, a)} (\mathcal{H}Q)(s', \omega) - \gamma \cdot \omega^\top \mathbb{E}_{s' \sim P(\cdot | s, a)} (\mathcal{H}Q')(s', \omega)| \\ &\leq \gamma \cdot \sup_{s' \in \mathcal{S}, \omega \in \Omega} \left| \omega^\top \left[\arg_Q \sup_{a' \in \mathcal{A}, \omega' \in \Omega} \omega^\top Q(s', a', \omega') - \arg_Q \sup_{a'' \in \mathcal{A}, \omega'' \in \Omega} \omega^\top Q'(s', a'', \omega'') \right] \right| \\ &\leq \gamma \cdot \sup_{s' \in \mathcal{S}, \omega \in \Omega} \left| \sup_{a' \in \mathcal{A}, \omega' \in \Omega} \omega^\top Q(s', a', \omega') - \sup_{a'' \in \mathcal{A}, \omega'' \in \Omega} \omega^\top Q'(s', a'', \omega'') \right| \end{aligned}$$

Step 2 to 3 is because $|\mathbb{E}[\cdot]| \leq \mathbb{E}[|\cdot|] \leq \sup |\cdot|$, and step 3 to 4 results from the cancellation between $\omega^\top$ and $\arg_Q$ (as justified above). According to our assumption, let a' and ω' be the action and preference chosen to maximize the value of $\omega^\top Q$ for some state s' and preference ω of interest, then we derive

$$\begin{aligned} d(\mathcal{T}Q, \mathcal{T}Q')(s, a) &\leq \gamma \cdot \sup_{s' \in \mathcal{S}, \omega \in \Omega} \left| \omega^\top Q(s', a', \omega') - \sup_{a'' \in \mathcal{A}, \omega'' \in \Omega} \omega^\top Q'(s', a'', \omega'') \right| \\ &= \gamma \cdot \sup_{s' \in \mathcal{S}, \omega \in \Omega} |\omega^\top Q(s', a', \omega') - \omega^\top Q'(s', a', \omega') \\ &\quad + \omega^\top Q'(s', a', \omega') - \sup_{a'' \in \mathcal{A}, \omega'' \in \Omega} \omega^\top Q'(s', a'', \omega'')| \\ &\leq \gamma \cdot \sup_{s' \in \mathcal{S}, \omega \in \Omega} |\omega^\top Q(s', a', \omega') - \omega^\top Q'(s', a', \omega')| \\ &\leq \gamma \cdot \sup_{\substack{s' \in \mathcal{S}, a' \in \mathcal{A} \\ \omega \in \Omega}} |\omega^\top Q(s', a', \omega') - \omega^\top Q'(s', a', \omega')| = \gamma d(Q, Q') \end{aligned}$$

The step 2 to 3 arises from the w.l.o.g. assumption that $\omega^\top Q(s', a', \omega') - \sup_{a'', \omega''} \omega^\top Q'(s', a'', \omega'') \geq 0$, as stated in lines 612 and 615. Thus, the whole expression in $|\cdot|$ is nonnegative and $\omega^\top Q(s', a', \omega') - \omega^\top Q'(s', a', \omega') \geq 0$. We can discard the last two terms since $\omega^\top Q'(s', a', \omega') \leq \sup_{a'', \omega''} \omega^\top Q'(s', a'', \omega'')$. Step 3 to 4 is because $\sup_{s', \omega'} f(s', a', \omega') \leq \sup_{s', a'', \omega''} f(s', a'', \omega'')$ holds for any a' and $f(\cdot)$.

This completes our proof that $\mathcal{T}$ is a contraction. $\square$

Remember that in our design, envelope version value distance d is a pseudo-metric. In a pseudo-metric space iteratively applying contraction may not shrink to the desired fixed point. To assert the convergence effectiveness of our design for optimality operator, we need to investigate a generalized Banach's Fixed-Point Theorem in the pseudo-metric space.

A.2.2 Generalized Banach's Fixed-Point Theorem

Theorem 3 (Generalized Banach Fixed-Point Theorem). *Given that $\mathcal{T}$ is a contraction mapping with Lipschitz coefficient γ on the complete pseudo-metric space $\langle \mathcal{Q}, d \rangle$, and $\mathbf{Q}^*$ is defined as that in Theorem 1, it is always true that $\lim_{n \rightarrow \infty} d(\mathcal{T}^n \mathbf{Q}, \mathbf{Q}^*) = 0$ for any $\mathbf{Q} \in \mathcal{Q}$.*

Proof. By the symmetry and triangle inequality of pseudo-metric, for any $\mathbf{Q}, \mathbf{Q}' \in \mathcal{Q}$,

$$\begin{aligned} d(\mathbf{Q}, \mathbf{Q}') &\leq d(\mathbf{Q}, \mathcal{T}\mathbf{Q}) + d(\mathcal{T}\mathbf{Q}, \mathcal{T}\mathbf{Q}') + d(\mathcal{T}\mathbf{Q}', \mathbf{Q}') \\ &\leq d(\mathbf{Q}, \mathcal{T}\mathbf{Q}) + \gamma d(\mathbf{Q}, \mathbf{Q}') + d(\mathcal{T}\mathbf{Q}', \mathbf{Q}') \\ \Rightarrow d(\mathbf{Q}, \mathbf{Q}') &\leq [d(\mathcal{T}\mathbf{Q}, \mathbf{Q}) + d(\mathcal{T}\mathbf{Q}', \mathbf{Q}')]/(1 - \gamma) \end{aligned}$$

Consider two points $\mathcal{T}^\ell \mathbf{Q}, \mathcal{T}^m \mathbf{Q}$ in the sequence $\{\mathcal{T}^n \mathbf{Q}\}$. Their distance is bounded by

$$\begin{aligned} d(\mathcal{T}^\ell \mathbf{Q}, \mathcal{T}^m \mathbf{Q}) &\leq [d(\mathcal{T}^{\ell+1} \mathbf{Q}, \mathcal{T}^\ell \mathbf{Q}) + d(\mathcal{T}^{m+1} \mathbf{Q}, \mathcal{T}^m \mathbf{Q})]/(1 - \gamma) \\ &\leq [\gamma^\ell d(\mathcal{T}\mathbf{Q}, \mathbf{Q}) + \gamma^m d(\mathcal{T}\mathbf{Q}, \mathbf{Q})]/(1 - \gamma) \\ &\leq \frac{\gamma^\ell + \gamma^m}{(1 - \gamma)} d(\mathcal{T}\mathbf{Q}, \mathbf{Q}) \end{aligned}$$

since $\gamma \in [0, 1)$ the distance $d(\mathcal{T}^\ell \mathbf{Q}, \mathcal{T}^m \mathbf{Q})$ converge to 0 as $\ell, m \rightarrow \infty$, proving that $\{\mathcal{T}^n \mathbf{Q}\}$ is a Cauchy sequence. Because $\langle \mathcal{Q}, d \rangle$ is a complete pseudo-metric space, $\lim_{n \rightarrow \infty} d(\mathcal{T}^n \mathbf{Q}, \mathbf{Q}^\diamond) = 0$ for some $\mathbf{Q}^\diamond \in \mathcal{Q}$. Therefore,

$$d(\mathcal{T}\mathbf{Q}^\diamond, \mathbf{Q}^\diamond) = \lim_{n \rightarrow \infty} d(\mathcal{T}^{n+1} \mathbf{Q}, \mathbf{Q}^\diamond) = \lim_{n \rightarrow \infty} d(\mathcal{T}^n \mathbf{Q}, \mathbf{Q}^\diamond) = 0$$

We claim that $\mathbf{Q}^\diamond$ and $\mathbf{Q}^*$ must lie in the same equivalent class partitioned by relation $d(\cdot, \cdot) = 0$. Suppose $d(\mathbf{Q}^\diamond, \mathbf{Q}^*) \neq 0$, then we can get a contradiction

$$\begin{aligned} d(\mathbf{Q}^\diamond, \mathbf{Q}^*) &= d(\mathbf{Q}^\diamond, \mathcal{T}\mathbf{Q}^\diamond) + d(\mathcal{T}\mathbf{Q}^\diamond, \mathcal{T}\mathbf{Q}^*) + d(\mathcal{T}\mathbf{Q}^*, \mathbf{Q}^*) \\ &\leq 0 + \gamma d(\mathbf{Q}^\diamond, \mathbf{Q}^*) + 0 \\ &< d(\mathbf{Q}^\diamond, \mathbf{Q}^*) \end{aligned}$$

This proves our claim. Therefore, $\lim_{n \rightarrow \infty} d(\mathcal{T}^n \mathbf{Q}, \mathbf{Q}^*) = 0$ for any $\mathbf{Q} \in \mathcal{Q}$. $\square$

In other words, Theorem 3 guarantees that iteratively applying optimal operator $\mathcal{T}$ on any multi-objective Q-value function, the algorithm will terminate with a function $\mathbf{Q}^\diamond$ which is equivalent to $\mathbf{Q}^*$ under the measurement of pseudo-metric d . Actually, these $\mathbf{Q}^\diamond$'s are as good as $\mathbf{Q}^*$, since they all have the same utilities for each ω , and only differ in the real value when the utility corresponds a recess in the utility control frontier.

A.2.3 Updating Scheme

Hindsight Experience Replay (HER) Paper [11] presents a technique for training a reinforcement learning to serve multiple goals. For each episode, the agent performs following a policy according to a randomly sampled goals. When updating, the agent uses the past trajectory update for multiple other goals in parallel. They referred this method as *hindsight experience replay* (HER). Though our settings are completely different, a similar method can be employed to update utility-based multi-objective Q-network here.

In the learning phase of the unknown linear preference scenario, for each training episode, the MORL agent randomly sample a preference ω from a certain distribution $\mathcal{D}_\omega$. When updating the multi-objective Q-network accordingly, for each sampled transition record $(s_t^i, a_t^i, r_t^i, s_{t+1}^i)$ from replay buffer $\mathcal{D}_\tau$, we associate it with N_ω preferences $\{\omega_1, \omega_2, \dots, \omega_{N_\omega}\}$ sampled from $\mathcal{D}_\omega$. The update is applied to an expended batch of size $\text{minibatch_size} \times N_\omega$. Note that the preferences sampled for actions only influence the agent's actions, not the environment dynamics. In this way, the trajectories can be replayed with arbitrary preferences with "hindsight".

Homotopy Optimization We use deep neural networks to approximate bounded functions in $\mathcal{Q} \subseteq (\Omega \rightarrow \mathbb{R}^m)^{\mathcal{S} \times \mathcal{A}}$ with parameters θ . We refer to this neural network as a *multi-objective Q-network*. To drag $\mathbf{Q}$ close to $\mathcal{T}\mathbf{Q}$ at each update step, the multi-objective Q-network can be trained by minimizing a series of loss functions

$$L_k^A(\theta) = \mathbb{E}_{s,a,\omega} [\|\mathbf{y}_k - \mathbf{Q}(s, a, \omega; \theta)\|_2^2], \quad (21)$$

which changes at each iteration k , where $\mathbf{y}_k = \mathbb{E}_{s'} [\mathbf{r}(s, a) + \gamma(\mathcal{H}\mathbf{Q})(s', a', \omega; \theta_k)]$ is the target of iteration k . The target is fixed during optimizing this loss function.

Minimizing the loss function L^A is trying to drag the vector $\mathbf{Q}$ close to $\mathcal{T}\mathbf{Q}$. This ensures the correctness of our algorithm, to predict a $\mathbf{Q}$ as the real solution, while this means the square error is hard to be optimized in practice. To address this problem, we use a sequence of auxiliary loss function L^B to directly optimized the value metric d , which is defined by

$$L_k^B(\theta) = \mathbb{E}_{s,a,\omega} [\|\omega^\top \mathbf{y}_k - \omega^\top \mathbf{Q}(s, a, \omega; \theta)\|] \quad (22)$$

Our final loss function sequence $L_k(\theta) = (1 - \lambda_k) \cdot L_k^A(\theta) + \lambda_k \cdot L_k^B(\theta)$, where λ_k is a weight to trade off between losses L_k^A and L_k^B . We increase the value of λ_k from 0 to 1, to shift our loss function from L^A to L^B . This method called *homotopy optimization* [12] is effective since for each update step, it uses the optimization result from the previous step as the initial guess. In the envelope deep MORL algorithm, L^A first ensure the prediction of Q is close to any real expected total reward, though it is hard to be optimal. And then L^B can provide an auxiliary force to pull the current guess along the direction with better utility. Figure 6 illustrate an explanation for this homotopy optimization.

The parameters of the multi-objective Q-network will be updated by $\theta_{k+1} \leftarrow \theta_k - \eta$, where

$$\eta \propto \nabla_{\theta=\theta_k} L_k(\theta) = -\mathbb{E}_{s,a,s'} [(\mathbf{r} + \gamma(\mathcal{H}\mathbf{Q})(s', a', \omega; \theta_k) - \mathbf{Q}(s, a, \omega; \theta_k))^T \nabla_{\theta=\theta_k} \mathbf{Q}(s, a, \omega; \theta)] . \quad (23)$$

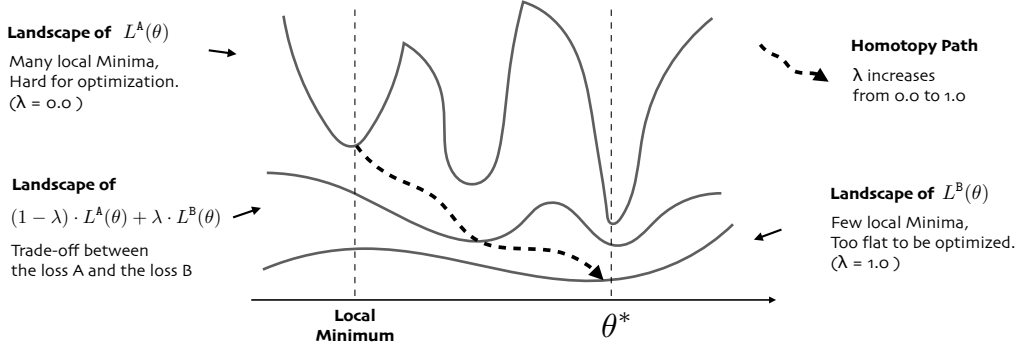


Figure 6: An explanation for homotopy optimization method used in the envelope deep MORL algorithm. The MSE loss L^A is hard for optimization since there are many local minima over its landscape. Although the value metric loss L^B has fewer local minima, it is also hard for optimization since there are many vectors Q minimizing value metric d . The landscape of L^B is too flat. The homotopy path connecting L^A and L^B provides better opportunities to find the global optimal parameters θ^*

When updating, we sample a minibatch of transition records from this replay buffer with HER. Theorems 1-3 and 5 guarantees the convergence of this minibatch updating, with an extra assumption that we can update the Q-function according to equation 23 for each $\omega \in \Omega$ infinite times. We use *hindsight experience replay* (HER) to ensure this. Notice that we will apply our optimality filter on the HER expended batch. Therefore the cost of solving the convex envelope is acceptable. Our multi-objective Q-network can also be replaced with other models similar to those in single-objective off-policy RL algorithms. In the experiment, we also use some popular deep reinforcement learning techniques to stabilize and speed up our algorithms. The skeleton of our envelope deep MORL is shown as Algorithm 1.

B Experimental Details

We first demonstrate our experimental results on two synthetic domains, Deep Sea Treasure (DST) and Fruit Tree Navigation (FTN), as well as two complex real domains, Task-Oriented Dialog Policy Learning (Dialog) and SuperMario Game (SuperMario). We also elaborate specific model architecture and implementation details in this section.

B.1 Domain Details

Deep Sea Treasure (DST) Our first experiment domain is a grid-world navigation problem, *Deep Sea Treasure*. This episodic problem was first explicitly created to highlight the limitations of linear scalarization [14]. However, in this paper, we use this environment as a delayed linear preference scenario. We ensure the Pareto frontier of this environment is convex, therefore the Pareto frontier itself is the its CCS.

In DST, an agent controls a submarine searching for treasures in a 10×11 -grid world while trading off **time-cost** and **treasure-value**. The grid world contains 10 treasures of different values. Their values increase as their distances from the starting point $s_0 = (0, 0)$ increase. An agent’s action spaces are formed by navigation in four directions. The reward has two dimensions: the first dimension indicates a time penalty, which is -1 on all turns; and the second dimension is the treasure value which is 0 except when the agent moves into a treasure location. We ensure the Pareto frontier of this environment to be convex. We depicted the map in Figure 7.

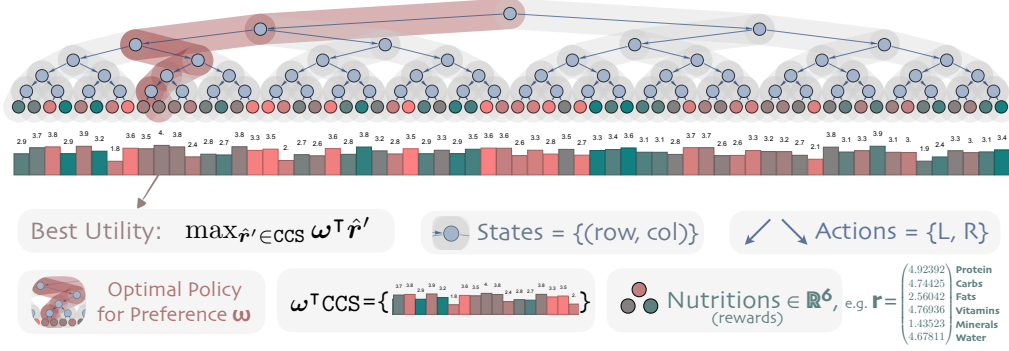


Figure 8: Fruit Tree Navigation (FTN): An agent travels from the root node to one of the leaf node to pick a fruit according to a post-assigned preference ω on the components of nutrition, treated as different objectives. The observation of an agent is its current coordinates (row, col), and its valid actions are moving to the left or the right subtree.

Fruit Tree Navigation (FTN) Our second experiment domain is a full binary tree of depth d with randomly assigned vectorial reward $\mathbf{r} \in \mathbb{R}^6$ on the leaf nodes. These rewards encode the amounts of six different components of nutrition of the *fruits* on the tree: {Protein, Carbs, Fats, Vitamins, Minerals, Water}. For every leaf node, $\exists \omega$ for which its reward is optimal, thus all leaves lie on the CCS. The goal of our MORL agent is to find a path from the root to a leaf node that maximizes utility for a given preference, choosing between left or right subtrees at every non-terminal node.

Figure 8 shows an instance of the *fruit tree navigation* task when $d = 6$, in which every non-leaf node is associated with zero reward and every fruit is a potential optimal solution in the convex cover set of the Pareto frontier. To construct this, we sample $\mathbf{r}^{(i)} = (\mathbf{v}_+^{(i)} + \mathbf{v}_-^{(i)}) / \|\mathbf{v}_+^{(i)}\|_2$, where $\mathbf{v}^{(i)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, for each fruit i on a leaf node. The optimal multiple-policy model for this tree structured MOMDP should contain all the paths from the root to different desired fruits. In experiments, we also test on $d = 5$ and $d = 7$ cases.

In this multi-objective environment, an optimal policy can be easily learned if we know the preference function $f_\omega(\cdot) = \langle \omega, \cdot \rangle$ for scalarization. However, since here we are interested in evaluating whether a multiple-policy neural network, trained with deep MORL algorithms, can find and maintain all the potential optimal policies (i.e., paths to every leaf node) when the preference function is unknown, and adapt to the optimal policy when a specific preference is given or hidden during execution.

Task-Oriented Dialog Policy Learning (Dialog) Our third experimental domain is a modified task-oriented dialog system in the restaurant reservation domain based on PyDial [36], where an agenda-base user simulator [43] with an error model to simulate the recognition and understanding errors arisen in the real system due to in the speech noise and ambiguity. A. We consider the task success rate and the dialog brevity (measured by number of turns) as two competing objectives of this domain.

Finding a good trade-off between multiple potentially competing objectives is usually domain-specific and not straightforward. For example, in the case when the objectives are brevity and success, if the relative importance weight for success is too high, the resulting policy is insensitive to potentially annoying actions such as `repeat()` provided that the dialogue is eventually successful. In this case, the obtained optimal policy cannot fit all users' preference, and sometimes is out of our expectation. Adaptation to user preferences and balancing these objectives is rarely considered.

In the standard single-objective reinforcement learning formulation, the goal of the policy model is to interact with a human user by choosing actions in each turn to maximize future rewards. We define the dialogue state shared by dialogue state tracker in the t -th turn as state s_t . The action taken by policy model under current

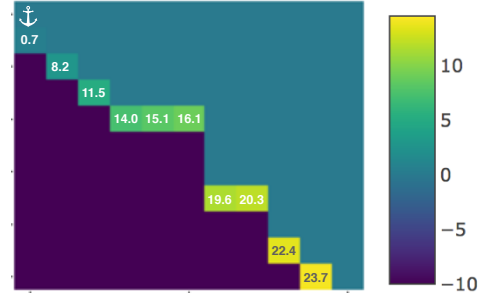


Figure 7: Deep Sea Treasure (DST): An agent controls a submarine searching for treasures in a 10×11 -grid world. The state s_t consists of the agent's current coordinates (x, y) . An agent's action spaces is navigation in four directions. The reward received by the agent is a 2-dimensional vector (time penalty, treasure value).

policy π_θ with parameters θ in the t -th turn as a_t , and $a_t \sim \pi(\cdot|s_t)$. The stochastic transition kernel is unknown but determined by human users or user simulators. In an ideal dialogue environment, once the policy model emits an action a_t , the human user will give an explicit feedback, like a normal response or a feedback of whether the dialogue is successful, which will be converted to a reward signal r_t delivering to the policy model immediately, and then the policy model will transit to next state s_t . The reward signal is an average of values of two objectives, brevity and success, e.g., $r_t = 0.5r_t^{\text{turn}} + 0.5r_t^{\text{succ}}$. Typically, r_t^{turn} is fixed for each turn as a negative constant R^{turn} , while r_t^{succ} equals a positive constant R^{succ} only when the dialogue terminates and receives a successful user feedback otherwise it equals zero.

To address this problem, we transform the dialogue learning process into a MORL scenario with vectorized (2-D) rewards: $\mathbf{r}_t = [r_t^{\text{turn}} \quad r_t^{\text{succ}}]^\top$, where r_t^{turn} is a turn penalty for the brevity objective and r_t^{succ} is a reward provided on successful completion of the task. In the learning phase, the linear preference ω over these two objectives are unknown, while the computational resources are abundant. The task-oriented dialogue system needs to learn all the possible optimal policies with sampled ω 's (achieved by user simulators or collected interactions with real users). While in the adaptation phase, learning is unaffordable because of the limitation of resources. The task-oriented dialogue system needs to respond the user with a specified user preference. User's utility increase is aligned with the system's utility increase $\omega^\top \mathbf{r}_t$. Paper [44] proposes a structured method, which is equivalent to the scalarized baseline without hindsight experience replay, for finding the optimal weights for a multi-objective reward function.

As for more experimental details, our dialog domain is a restaurant reservation hotline which provides information about restaurants in Cambridge. There are 3 search constraints, 9 informational items that the user can request, and 110 database entities. The reward R^{turn} is -1 for each turn, and $R^{\text{succ}} = 20$. The maximal length of dialogue is 25. We apply our envelope deep MORL algorithm to this dialogue policy learning task, and compare to traditional single-objective methods and other baselines. All the single-objective and multi-objective reinforcement learning are trained for 3,000 sessions with 15% simulated speech recognition and understanding error rate.

Multi-Objective SuperMario Game (SuperMario) Our final environment is a version of the popular video game Super Mario Bros. We modify the open-source environment from OpenAI gym [37] to provide vectorized rewards encoding five different objectives: `x-pos`: value corresponding to the difference in Mario's horizontal position between current and last time point, `time`: a small negative time penalty, `deaths`: a large negative penalty given each time Mario dies³, `coin`: rewards for collecting coins, and `enemy`: rewards for eliminating an enemy. The state is a stack of four continuous frames of game images rendered by the simulator, and there are seven valid actions each step: { 'NOOP', 'right', 'right+A', 'right+B', 'right+A+B', 'A', 'left' }, where the button 'A' is used to jump and the button 'B' is used to run. We restrict the Mario to only play the stage I.

We use an A3C [38] variant of our envelope MORL algorithm. During the learning phase, the agent does not know the underlying preference, and hence needs to learn a multi-objective policy within 32k training episodes. During the adaptation phase, we test our agents under 500 uniformly random preferences and test the its preference elicitation ability (as described in Section 3) within 100 episodes to uncover the underlying preference that maximizes utility.

B.2 Implementation Details

Our multi-objective Q-network can be replaced with any model similar to that in single-objective off-policy RL algorithms like DDPG [45], NAF [46] or SDQN [47]. In the experiment, we use a variate of deep reinforcement learning techniques including double Q-learning [48] with a target network and prioritized experience replay [49], which stabilize and speed up our algorithms.

Architectures of the Multi-objective Q-Network We implement the Multi-objective Q-networks (MQNs) by 4 fully connected hidden layers with $\{16, 32, 64, 32\} \times (\dim(S) + m)$ hidden units respectively. The multi-objective Q-networks are similar to Deep Q-Networks (DQNs) [34], but differs on inputs. An input of the multi-objective Q-network is a concatenation of state representation and parameters of a linear preference function. The output layer of the scalarized MORL algorithm is of size $|\mathcal{A}|$, and that of envelope version is of size $m \times |\mathcal{A}|$. Here $\dim(S)$ is the dimensionality of the state space, $|\mathcal{A}|$ is the cardinality of the action set, and m is the number of objectives.

Multi-Objective A3C We use the multi-objective A3C (MoA3C) algorithms for Mario experiment. The skeleton of the envelope MoA3C algorithm is provided in Algorithm 2. In MoA3C Both critic and actor networks contain three shared convolutional layers for feature extraction from raw images input. The extracted features are then concatenated with preferences, and fed into two-layer fully connected networks for output. For the scalarized version MoA3C, the output of the critic network is just one-dimensional utility prediction, whereas the output of the envelope version critic network is m -dimensional returns prediction. Both scalarized

³Mario has up to three lives in one episode.

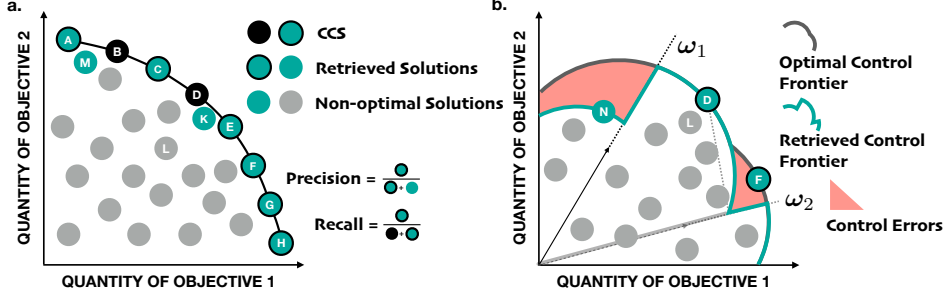


Figure 9: Quantitative evaluation metrics for multi-objective reinforcement learning. (a.) Coverage ratio measures an agent’s ability to find all the potential optimal solutions in the convex coverage set of Pareto frontier. (b.) Adaptation quality measures an agent’s ability of policy adaptation to real-time specified preferences.

and envelope versions have the same actor network architecture to output the probability distribution over the action space. We train them with 16 workers in parallel with different sampled preferences, and it take around 10 hours for the envelope version MoA3C to converge to a good level of performance.

Training with Prioritized Double Q-Learning When training the with our MORL algorithm on DST and FTN tasks, we employ techniques of *prioritized experience replay* [49] and *double Q-learning* [48] to speed up the training process and to yield more accurate value estimates. Double Q-Learning introduces a target network Q_{target} to replace the estimate of $TQ(s, a, \omega)$, $y_t(\omega) = \omega^\top r + \gamma Q_{\text{target}}(s', a, \omega)$ with $y_t^{\text{double}}(\omega) = \omega^\top r + \gamma Q_{\text{target}}(s', \arg \max_a Q(s', a, \omega), \omega)$ for scalarized version of algorithm, and similarly we replace $y_t(\omega) = \omega^\top r + \gamma \omega^\top Q_{\text{target}}(s', a, \omega)$ with $y_t^{\text{double}}(\omega) = \omega^\top r + \gamma \omega^\top Q_{\text{target}}(s', \arg \max_a \omega^\top Q(s', a, \omega), \omega)$ for envelope version of MORL algorithm. We update the target network by coping from Q-network every 100 steps. The priority of sampling transition $\tau_i = (s, a, r, s')$ is $p_i^{\text{scalarized}} = |y_t^{\text{double}}(\omega) - Q(s, a, \omega)|$ for the scalarized version of MORL algorithm, and similarly $p_i^{\text{envelope}} = |y_t^{\text{double}}(\omega) - \omega^\top Q(s, a, \omega)|$ for the envelope version of algorithm, where ω is sampled from the distribution $\mathcal{D}_\omega$. When updating the network, a trajectory is sampled by $\tau_i \sim P(i) = p_i / \sum_i p_i$. The replay memory size is 4000 and the batch size is 32. For the deep tree navigation task, we train each model for total 5000 episodes, and update it by Adam optimizer every step after at least a batch experiences are stored in replay buffer, with a learning rate $\text{lr} = 0.001$.

Training Details for Dialogue Policy Learning All the single-objective and multi-objective reinforcement learning are trained for 3,000 sessions. We evaluate learned policies on 5,000 sessions with randomly assigned user preferences. The preference distribution $\mathcal{D}_\omega$ is same as the one we used in previous deep sea treasure experiment (see Section 4.3.1), which is a nearly uniform distribution. For the single-objective reinforcement learning algorithms, we set three groups of ω as $\{(0.5, 0.5), (0.2, 0.8), (0.8, 0.2)\}$. For the envelope deep MORL algorithms, the homotopy path is a monotonically increasing track where λ increases from 0.0 to 1.0 exponentially. The number of sampled preferences N_ω is 32 for both scalarized and envelope deep MORL algorithms. The exploration policy used for training these reinforcement learning algorithms is ϵ -greedy, where $\epsilon = 0.5$ initially and then decays to zero linearly during the training process. For all the single-objective and multi-objective algorithms, we employ the same deep Q-network architecture, which comprises 3 fully connected hidden layers with $\{16, 32, 32\} \times (\dim(S) + m)$ hidden units. The minibatch size is 64 for all. An Adam optimizer is used for updating the parameters of all these algorithms with an initial learning rate $\text{lr} = 0.001$.

Computing Infrastructure We ran the synthetic experiments and the dialog experiments on a workstation with one GeForce GTX TITAN X GPU, 12 Intel(R) Core(TM) i7-5820K CPUs @ 3.30GHz, and 32G memory and ran the SuperMario experiments on a cluster with twenty 2080 RTX GPUs, 40 CPUs and 200GB memory.

C Additional Experimental Results

C.1 Evaluation Metrics

We design two metrics to evaluate the empirical performance of our algorithms on test tasks. Slightly different from the main article, we introduce adaptation quality (AQ) here other than adaptation error to adjust the value range and get a score in $(0, 1]$.

Coverage Ratio (CR). The first metric is *coverage ratio* (CR), which evaluates the agent’s ability to recover optimal solutions in the convex coverage set (CCS). If $\mathcal{F} \subseteq \mathbb{R}^m$ is the set of solutions found by the agent (via sampled trajectories), we define $\mathcal{F} \cap_\epsilon \text{CCS} := \{x \in \mathcal{F} \mid \exists y \in \text{CCS} \text{ s.t. } \|x - y\|_1 / \|y\|_1 \leq \epsilon\}$ as the intersection between these sets with a tolerance of ϵ . The CR is then defined as:

$$\text{CR}_{F1}(\mathcal{F}) = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}, \quad (24)$$

Algorithm 2: Envelope Multi-Objective A3C (EMoA3C) Algorithm - Pseudocode for each actor-learner thread

Input:

- a preference sampling distribution $\mathcal{D}_\omega$;
- minibatch sizes for transitions N_τ and for preference N_ω ;
- a multi-objective critic-network V parameterized by θ_v ;
- an actor-network π parameterized by θ_π ;
- a balance weight λ for critic losses L^A and L^B .

Initialize replay buffer $\mathcal{D}_\tau$.

for $episode = 1, \dots, M$ **do**

 Synchronize thread-specific parameters $\theta'_v = \theta_v$ and $\theta'_\pi = \theta_\pi$.

 Sample a linear preference $\omega \sim \mathcal{D}_\omega$.

for $t = 0, \dots, N - 1$ **do**

 Observe state s_t .

 Sample an action a_t using according to policy $\pi(a_t|s_t, \omega; \theta'_\pi)$.

 Receive a multi-objective reward $\mathbf{r}_t$ and observe new state s_{t+1} .

 Store transition $(s_t, a_t, \mathbf{r}_t, s_{t+1})$ in $\mathcal{D}_\tau$.

if *update* **then**

 Sample random minibatch of transitions $(s_j, a_j, \mathbf{r}_j, s_{j+1})$ from $\mathcal{D}_\tau$.

 Sample N_ω preferences $W = \{\omega_1, \omega_2, \dots, \omega_{N_\omega}\} \sim \mathcal{D}_\omega$.

 Compute

$$(\mathcal{TV})_{ij} = \begin{cases} \mathbf{r}_j & \text{for terminal } s_{j+1}; \\ \mathbf{r}_j + \gamma \arg_V \max_{a \in \mathcal{A}, \omega' \in W} \omega_i^\top V(s_{j+1}, \omega'; \theta) & \text{for non-terminal } s_{j+1}. \end{cases}$$

 Calculate $d\theta_v$ according to similar equations 6 and 7 w.r.t. θ'_v :

$$d\theta_v = (1 - \lambda) \cdot \nabla_{\theta'_v} L^A(\theta'_v) + \lambda \cdot \nabla_{\theta'_v} L^B(\theta'_v).$$

 Calculate $d\theta_\pi$ using the advantage w.r.t. θ'_v and θ'_π

$$d\theta_\pi = \frac{1}{N_\omega N_\tau} \sum_{i,j} \nabla_{\theta'_\pi} \log \pi(a_j|s_j, \omega_i; \theta'_\pi) [\omega_i^\top ((\mathcal{TV})_{ij} - V(s_j, \omega_i; \theta'_v))].$$

 Perform asynchronous update of θ_v using $d\theta_v$ and of θ_π using $d\theta_\pi$.

where the $\text{precision} = |\mathcal{F} \cap_\epsilon \text{CCS}|/|\mathcal{F}|$, indicating the fraction of optimal solutions among the retrieved solutions, and the $\text{recall} = |\mathcal{F} \cap_\epsilon \text{CCS}|/|\text{CCS}|$, indicating the fraction of optimal instances that have been retrieved over the total amount of optimal solutions (see Figure 3(a)). The F1 score is their harmonic mean. In our evaluation of both synthetic tasks DST and FTN, we set $\epsilon = 0.00$ for $\mathcal{F}_{\Pi_{\mathcal{L}}}$ (executive frontier) and $\epsilon = 0.20$ for $\mathcal{F}_Q$ (frontier predicted by Q-function). Figure 9 (a.) illustrates an example of the computation of coverage ratio.

Adaptation Quality (AQ). Our second metric compares the retrieved control frontier with the optimal one, when an agent is provided with a specific preference ω during the adaptation phase. The adaptation quality is defined by

$$\text{AQ}(\mathcal{C}) = \frac{1}{1 + \alpha \cdot \text{err}_{\mathcal{D}_\omega}}, \quad (25)$$

where the $\text{err}_{\mathcal{D}_\omega} = \mathbb{E}_{\omega \sim \mathcal{D}_\omega} [|C(\omega) - \mathcal{C}_{\text{opt}}(\omega)|/C_{\text{opt}}(\omega)]$ is the expected relative error between optimal control frontier $\mathcal{C}_{\text{opt}} : \Omega \rightarrow \mathbb{R}$ with $\omega \mapsto \max_{\hat{\mathbf{r}} \in \text{CCS}} \omega^\top \hat{\mathbf{r}}$ and the agent's control frontier $\mathcal{C}_{\pi_\omega} = \omega^\top \hat{\mathbf{r}}_{\pi_\omega}$, and α is a scaling coefficient to amplify the discrepancy.

Similarly, $\mathcal{C}_Q$ is the control frontier guessed by an agent (via predictions from Q-network) and we can compute the predictive $\text{AQ}(\mathcal{C}_Q)$ to evaluate the quality of multi-objective Q-network on the value prediction accuracy.

In all experiment domains, we use Gaussian distributions which are restricted to be positive part and ℓ_1 -normalized as our $\mathcal{D}_\omega$. We set $\alpha = 0.01$ for the DST task (because the penalty range is large), and $\alpha = 10.0$ for the FTN task (because the value differences are small). Figure 9 (b.) shows examples of optimal control frontier, retrieved control frontier, and the control discrepancy. Overall, CR provides an indication of agent's ability to learn the space of optimal policies in the learning phase, while AE tests its ability to adapt to new scenarios.

C.2 Deep Sea Treasure (DST)

We show more experimental results on DST tasks in this section. We train all agent for 2000 episodes. After training in the learning phase, our envelope MORL algorithms find all the potential optimal solutions and their corresponding policies.

Figure 10 presents the real CCS and the retrieved solutions of a MORL algorithm. The scalarized and envelope algorithm can find all the whole CCS. Figure 10 (b.) illustrates the real control frontier (the blue curve), retrieved control frontier (the green curve), and the predicted control frontier (the orange line). The retrieved control frontier is almost overlapped with the real control frontier, which indicates that the alignment between preferences and optimal policies is perfectly well. The agent can respond any given preference with the policy resulting in best utility.

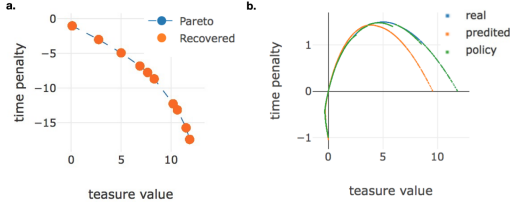


Figure 10: The solutions and control frontier found by MORL algorithm in the deep sea treasure task. (a.) The real CCS and the retrieved solutions. (b.) The real control frontier, predicted control frontier, and retrieved control frontier in the adaptation phase.

Method	DST		
	CR F1	Exe-AQ ($\alpha = 0.01$)	Pred-AQ ($\alpha = 0.01$)
MOFQI	0.639 ± 0.421	0.417 ± 0.134	0.226 ± 0.138
CN+OLS	0.751 ± 0.163	0.743 ± 0.008	0.177 ± 0.089
Scalarized	0.989 ± 0.024	0.998 ± 0.001	0.950 ± 0.034
Envelope	0.994 ± 0.001	0.998 ± 0.000	0.850 ± 0.045

Table 4: Performance comparison of different MORL algorithm in learning and adaptation phases on the DST environment. The Exe-AQ is the AQ measured on the real trajectories in the adaptation phase, and the Pred-AQ is the AQ values measured on the predictions of Q functions.

Table 4 provides the coverage ratio (CR) and adaptation quality (AQ) comparisons of different MORL algorithms. We trained all the algorithm in 2000 episodes and test for another 2000 episodes. Each data point in the table is an average of 5 train and test trails. For the CN+OLS baseline, we allow it to iterate for 25 corner weights. The envelope algorithm achieves best CR and execution AQ, and the scalarsized algorithm achieves best predictive adaption quality. Note that traditional evaluations rarely test the algorithm’s ability in learning phase and adaptation phase separately. Thus our setting is more challenging.

The classical deep sea treasure task shows the effectiveness of our deep multi-objective reinforcement learning algorithms, while it is relatively easy for the agent to find all the good policies. It only contains 10 potentially optimal solutions in the real CCS, therefore scalarized algorithm can efficiently solve this problem.

C.3 Fruit Tree Navigation (FTN)

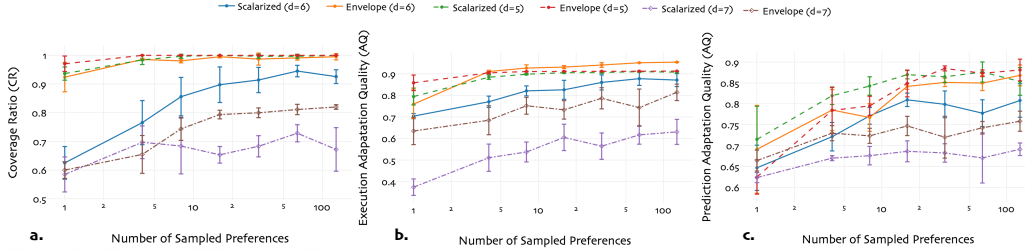


Figure 11: Coverage Ratio (CR) and Adaptation Quality (AQ) comparison of the scalarized deep MORL algorithm and the envelope deep MORL algorithm tested on fruit tree navigation tasks of depths $d = 5, 6, 7$. Trained on 5000 episodes and test on 2000 and 5000 episode to estimate CR and AQ, respectively. Each data point in the figure is an average of 5 trails of training and test.

Sample Efficiency To compare sample efficiency during the learning phase, we train envelope MORL algorithm and baselines on the FTN task of depth 5, 6, 7 for 5000 episodes. We compute coverage ratio (CR) over 2000 episodes and adaptation quality (AQ) over 5000 episodes. Figure 11 (blue and orange curves for $d = 6$) shows plots for the metrics computed over a varying number of sampled preferences N_ω from 1 to 128. When $N_\omega = 1$, both algorithms only update under single preference each time, therefore runs fast while it makes wasteful use of interactions. When $N_\omega = 128$, both algorithm needs to update for 128 sampled preferences in a batch. In this case the algorithms run slowly, while can make better use of interactions. Each point on the curve is averaged over 5 experiments. We observe that the envelope MORL algorithm consistently has a better CR and AQ scores than the scalarized baseline, with smaller variances. As N_ω increases, CR and AQ both increase,

which shows better use of historical interactions for both algorithms when N_ω is larger. This reinforces our theoretical analysis that the envelope MORL algorithm has better sample efficiency than the scalarized baseline.

Table 5 compares their coverage ratios. Each entry in the table is an average of 5 experiments (train and test). Due to the property of FTN task that every solution is potentially optimal, the CR precision is always 1 for both scalarized and envelope algorithm. While for the recall, which indicates the ability to find unseen optimal solutions in the learning phase, the envelope deep MORL algorithm is better than the scalarized version for all numbers of sampled preferences, and has relatively lower variances. Therefore the same to F1 scores. As N_ω increases, the CR value increases for both scalarized version and envelope version algorithms, which verifies a better use of historical interactions for both algorithm when N_ω is larger. As here the initial performance for the envelope version algorithm is good enough, it suddenly surpasses 0.98 of CR F1 score when it can sample more than one preference each update.

N_ω	Execution AQ		Prediction AQ		CR F1	
	Scalarized	Envelope	Scalarized	Envelope	Scalarized	Envelope
1	0.7037 $\pm$ 0.012	0.759 $\pm$ 0.066	0.6474 $\pm$ 0.054	0.6915 $\pm$ 0.105	0.625 $\pm$ 0.057	0.924 $\pm$ 0.051
4	0.7701 $\pm$ 0.026	0.9101 $\pm$ 0.006	0.7216 $\pm$ 0.034	0.7853 $\pm$ 0.049	0.7654 $\pm$ 0.077	0.9856 $\pm$ 0.004
8	0.8205 $\pm$ 0.023	0.9261 $\pm$ 0.015	0.7714 $\pm$ 0.02	0.7675 $\pm$ 0.033	0.856 $\pm$ 0.067	0.9808 $\pm$ 0.007
16	0.8255 $\pm$ 0.044	0.9306 $\pm$ 0.007	0.8097 $\pm$ 0.016	0.8417 $\pm$ 0.007	0.8976 $\pm$ 0.062	0.9952 $\pm$ 0.004
32	0.8597 $\pm$ 0.035	0.9402 $\pm$ 0.011	0.7989 $\pm$ 0.032	0.8513 $\pm$ 0.01	0.914 $\pm$ 0.044	0.987 $\pm$ 0.021
64	0.877 $\pm$ 0.031	0.9506 $\pm$ 0.001	0.7778 $\pm$ 0.032	0.8497 $\pm$ 0.018	0.9452 $\pm$ 0.02	0.9904 $\pm$ 0.007
128	0.8705 $\pm$ 0.03	0.9536 $\pm$ 0.002	0.8081 $\pm$ 0.04	0.868 $\pm$ 0.025	0.9258 $\pm$ 0.024	0.9952 $\pm$ 0.011

Table 5: Sample Efficiency - Coverage Ratio (CR) and Adaptation Quality (AQ) comparison of the scalarized deep MORL algorithm and the envelope deep MORL algorithm tested on fruit tree navigation task, where the tree depth $d = 6$. Trained on 5000 episodes.

N_ω	Execution AQ		Prediction AQ		CR F1	
	Scalarized	Envelope	Scalarized	Envelope	Scalarized	Envelope
1	0.7943 $\pm$ 0.039	0.8578 $\pm$ 0.036	0.7153 $\pm$ 0.079	0.6254 $\pm$ 0.041	0.9364 $\pm$ 0.023	0.9706 $\pm$ 0.027
4	0.8836 $\pm$ 0.01	0.9041 $\pm$ 0.005	0.8195 $\pm$ 0.021	0.7843 $\pm$ 0.055	0.984 $\pm$ 0.016	1 $\pm$ 0
8	0.8975 $\pm$ 0.003	0.9099 $\pm$ 0.001	0.8427 $\pm$ 0.022	0.7952 $\pm$ 0.023	0.9968 $\pm$ 0.007	1 $\pm$ 0
16	0.9047 $\pm$ 0.003	0.9109 $\pm$ 0.001	0.8698 $\pm$ 0.01	0.8488 $\pm$ 0.032	1 $\pm$ 0	1 $\pm$ 0
32	0.9054 $\pm$ 0.003	0.9113 $\pm$ 0.001	0.8647 $\pm$ 0.014	0.8847 $\pm$ 0.006	0.9968 $\pm$ 0.007	1 $\pm$ 0
64	0.9096 $\pm$ 0.003	0.9119 $\pm$ 0	0.8761 $\pm$ 0.024	0.8731 $\pm$ 0.008	0.9968 $\pm$ 0.007	1 $\pm$ 0
128	0.9071 $\pm$ 0.004	0.9121 $\pm$ 0	0.8535 $\pm$ 0.033	0.8809 $\pm$ 0.026	1 $\pm$ 0	1 $\pm$ 0

Table 6: Sample Efficiency when $d = 5$ - Coverage Ratio (CR) and Adaptation Quality (AQ) comparison of the scalarized and the envelope deep MORL algorithms tested on fruit tree navigation task, where the tree depth $d = 5$. Trained on 5000 episodes.

Table 5 also compares MORL algorithms' adaptation quality in the adaptation phase. Each entry in the table is an average of 5 experiments (train and test). For all numbers of sampled preferences, the envelope deep MORL algorithm has better execution AQ than the scalarized algorithm, in spite of better CR of the envelope version, it also indicates the envelope version algorithm has better alignment between preferences and policies. As N_ω increases, the values of execution AQ and prediction AQ of both algorithms keep increase. Note that even though when $N_\omega = 1$ the execution AQ of the scalarized version and the envelope version differs only around 0.5, when N_ω increase to 4, the envelope version algorithm better utilizes the sampled preferences to improve the execution AQ to above 0.9. This agrees with our theoretical analysis that our envelope deep MORL algorithm has better sample efficiency than the scalarized version.

We also investigate how the size of optimality frontiers will affect the performance of our algorithms. We train our deep MORL algorithms on two new FTN environments with $d = 5$ and $d = 7$ respectively. One is smaller than the previous environment, which contains only 32 optimal solutions, the other is larger than the previous environment, containing 128 solutions on the CCS. We fix the number of episode for training as 5000, and test 2000 episode to obtain coverage ratio, and test 5000 episode for policy adaptation quality.

Table 6 shows the results of coverage ratio evaluation in the environment of tree depth $d = 5$. As it shows, both scalarized and envelope algorithms work well in that environment. the CR F1 scores are very close to 1. The envelope version algorithm is more stable than the scalarized version deep MORL algorithm. Besides, the envelope deep MORL algorithm can predict multi-objective solutions, while the scalarized algorithm cannot. The prediction ability will also be improved as N_ω increases.

Visualizing Frontiers We also provide a visualization of the convex coverage set (CCS) and control frontier for our envelope algorithm and the scalarized baseline in Figure 12. The left figure shows the real CCS and retrieved CCS of both MORL algorithms using t-SNE [50]. We observe that the envelope algorithm (green dots) almost completely covers the entire set of optimal solutions in the real CCS whereas the scalarized algorithm (pink dots) does not. The right figure presents the slices of optimal control frontier and the control frontier of our algorithms along the Minerals-Water plane. The envelope version retrieves almost the entire optimal frontier,

N_ω	Execution AQ		Prediction AQ		CR FI	
	Scalarized	Envelope	Scalarized	Envelope	Scalarized	Envelope
1	0.3748 ± 0.038	0.6348 ± 0.063	0.624 ± 0.012	0.6646 ± 0.025	0.5847 ± 0.061	0.60 ± 0.029
4	0.5112 ± 0.063	0.6846 ± 0.067	0.6704 ± 0.006	0.730 ± 0.018	0.6969 ± 0.057	0.6544 ± 0.066
8	0.5376 ± 0.046	0.7516 ± 0.039	0.6763 ± 0.022	0.7237 ± 0.018	0.6837 ± 0.097	0.7437 ± 0.04
16	0.6052 ± 0.061	0.7328 ± 0.043	0.6867 ± 0.025	0.7473 ± 0.023	0.6532 ± 0.029	0.7936 ± 0.015
32	0.5646 ± 0.061	0.7862 ± 0.049	0.6828 ± 0.022	0.7204 ± 0.05	0.6829 ± 0.037	0.7997 ± 0.017
64	0.6168 ± 0.043	0.743 ± 0.086	0.671 ± 0.06	0.7429 ± 0.015	0.7284 ± 0.03	0.8112 ± 0.018
128	0.6308 ± 0.058	0.8138 ± 0.038	0.6916 ± 0.015	0.7586 ± 0.024	0.6719 ± 0.076	0.8199 ± 0.008

Table 7: Sample Efficiency when $d = 7$ - Coverage Ratio (CR) and Adaptation Quality (AQ) comparison of the scalarized and the envelope deep MORL algorithms tested on fruit tree navigation task, where the tree depth $d = 7$. Trained on 5000 episodes.

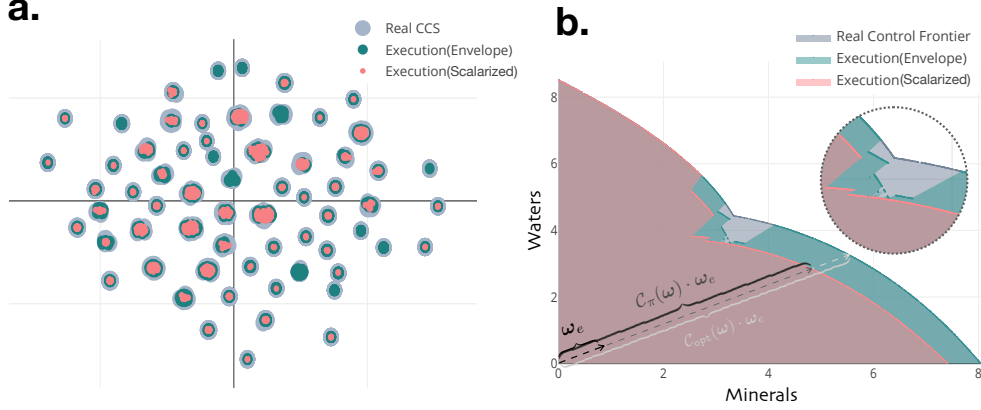


Figure 12: Comparison of CCS and control frontiers of deep MORL algorithms. The left figure (a) is visualizing the real CCS and retrieved CCS of scalarized and envelope MORL algorithms using t-SNE. The right figure (b) presents the slices of optimal control frontier and the control frontier of scalarized and envelope MORL algorithms along the Mineral-Waters plane.

while the scalarized version algorithm has larger control discrepancies. The small discrepancy between the real control frontier and the one retrieved by envelope version algorithm at the indentation of the frontier indicates an alignment issue between the preferences and optimal policies.

C.4 Task-Oriented Dialog Policy Learning (Dialog)

	Single-(0.5,0.5)	Single-(0.2,0.8)	Single-(0.8,0.2)	Scalarized	Envelope
SR	88.18 ± 0.90	85.30 ± 0.98	87.62 ± 0.91	86.38 ± 0.95	89.52 ± 0.85
#T	8.93 ± 0.13	9.40 ± 0.16	7.42 ± 0.10	8.08 ± 0.12	8.08 ± 0.12
UT	2.13 ± 0.23	1.84 ± 0.23	2.53 ± 0.22	2.38 ± 0.22	2.65 ± 0.22
AQ	0.660	0.279	0.728	0.614	0.814

Table 8: The average success rate (SR), number of turns (#T), user utility (UT), and adaptation quality (AQ, $\alpha = 0.1$) of policies obtained by single-objective RL baselines and two MORL algorithms.

Table 8 shows that envelope MORL algorithm achieves best success rate (SR) on average, while the single-objective method with equal weight on both dimensions (0.5) achieves competitive performance. However, on metrics of average utility (UT) and adaptation quality (AQ) (with $\alpha = 0.1$, envelope MORL is significantly better than the other methods, including scalarized MORL. This again demonstrates that adaptability of envelope MORL to tasks with new preferences. The single-objective method with 0.8 success weight has the worst performance on success rate eventually. This is might because lack of turn level reward guidance directly optimize for success is very difficult for the single-objective RL learner.

Figure 13 illustrates utility-weight curves during a policy adaptation phase with given preferences, where each data point is a moving average of closest 500 dialogues in the interval of around ± 0.05 weight of success over three trained policies. We find the envelope deep MORL algorithm is almost always better than other methods in terms of utility under certain given preferences, and the scalarized MORL baseline keeps a good level of

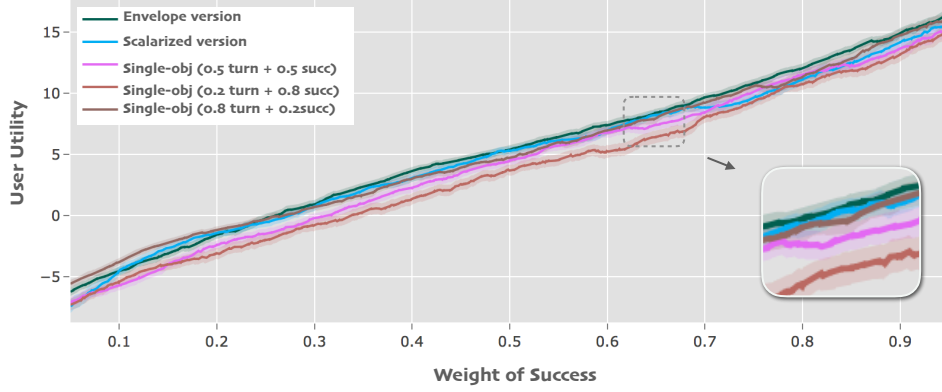


Figure 13: Utility-weight curves for the MORL and single-objective RL dialog policy learning after 3,000 training dialogues. We evaluate policies on 5,000 dialogues with near-uniformly randomly sampled preference. For each curve, each data point is a moving average of the closest 500 dialogues in the interval of around ± 0.05 weight of success over three trained policies.

utility under almost all user preferences. Single-objective RL algorithms are good only when the user’s weight of success is close to their fixed preferences while training.

C.5 Multi-Objective SuperMario Game (SuperMario)

Method	Super Mario		
	Avg. UT (0.5 x-pos & 0.5 time)	Avg. UT (0.5 coin & 0.5 enemy)	Avg. UT (uniform preference)
Scalarized	317.1 $\pm$ 123.7	76.7 $\pm$ 36.5	301.7 $\pm$ 49.2
Envelope	600.9 $\pm$ 114.9	233.3 $\pm$ 31.2	319.7 $\pm$ 34.4

Table 9: Performance comparison of different MORL algorithm in learning and adaptation phases on the SuperMario environment under three different preferences. The first preference emphasizes the fast completion of the task so it is 0.5 on x-pos and 0.5 time, the second preference emphasizes collecting coin and eliminating enemy. The third is a uniform preference which has weights 0.2 for all five objectives {x-pos, time, death, coin, enemy}

We compared the average utility of the scalarized baseline and the MORL algorithm with envelope update. The first preference emphasizes the fast completion of the task so it is 0.5 on x-pos and 0.5 time, the second preference emphasizes collecting coin and eliminating enemy. The third is a uniform preference which has weights 0.2 for all five objectives {x-pos, time, death, coin, enemy}. The envelope algorithm outperforms the scalarized algorithm under all these preferences.

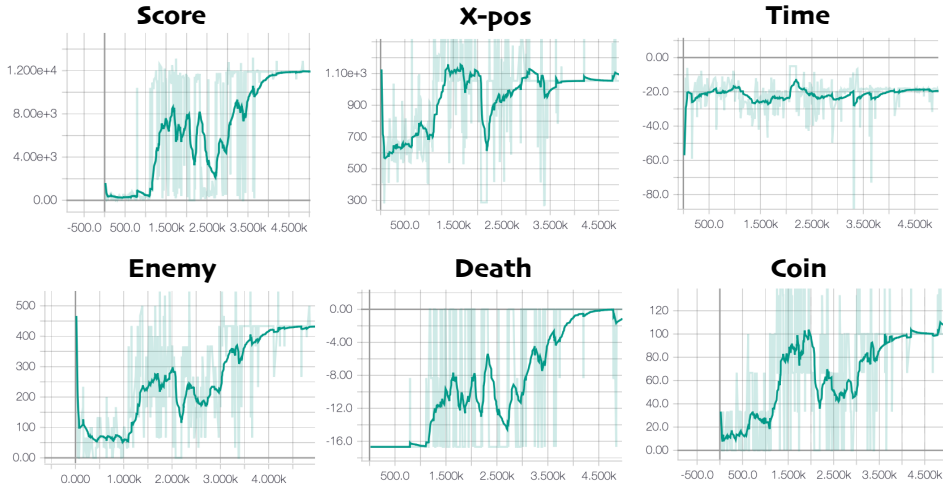


Figure 14: The training curves of the Envelope Multi-Objective A3C (EMoA3C) algorithm.

Table 10: Inferred preferences of the scalarized multi-objective A3C algorithm in different Mario Game variants with 100 episodes.

	x-pos	time	life	coin	enemy
g1	0.2315	0.2569	0.335	0.0522	0.1243
g2	0.1600	0.1408	0.2822	0.1955	0.2216
g3	0.1865	0.1837	0.0874	0.3759	0.1665
g4	0.1250	0.3062	0.3522	0.1240	0.0926
g5	0.2378	0.1673	0.1820	0.3432	0.0698

Table 11: The difference between the inferred preferences of the envelope and the scalarized multi-objective A3C algorithms.

	x-pos	time	life	coin	enemy
g1	+0.2973	-0.0799	-0.1850	-0.0052	-0.0271
g2	+0.0385	+0.0829	-0.0337	-0.0533	-0.0348
g3	+0.0331	-0.0541	+0.2667	-0.1967	-0.0490
g4	-0.1039	-0.0658	-0.3311	+0.5720	-0.0715
g5	-0.1663	-0.0635	+0.0249	+0.0490	+0.1555

The training curves of EMOA3C algorithm are shown in Figure 14. To plot Figure 14, we use a uniform preference $[0.2 \ 0.2 \ 0.2 \ 0.2 \ 0.2]$ as the probe preference to sample trajectories for evaluation. We observe the algorithm converges within around 5k episodes of training.

Finally we compare the difference between the inferred preferences of the envelope and the scalarized multi-objective A3C algorithm on different variants of SuperMario game, g1 to g5, where only corresponding scalar rewards are available. We only consider maximizing one objective in each game variant, because this orthogonal design helps us characterize the behaviors of agents and compare their inferred preferences. We allow agents to make 100 episodes interactions in each game variance, to determine the preference. Note that only 100 episodes are far from enough for training a single objective reinforcement learning agent, even the model is pre-trained on other tasks.

As Table 11 illustrates, the preferences inferred by the envelope MoA3C agent is more concentrate on the diagonal than that of scalarized MoA3C agent, which is more closer to the true underlying preferences.

Even though the EMOA3C agent can do better, our experiments show that the inferred preferences are not exactly the true underlying preferences. It is mainly for two reasons: First, the trade-off frontiers and policy-preference alignment learned by the algorithm is not ideal. There might be some discrepancies between the obtained control frontier and the real optimal control frontier just like what Figure 12 (b) illustrates. Second, even the agent perfectly learned the frontier and the alignment, close preferences may correspond to the policies with the same expected returns.

We also deploy the trained EMOA3C agents in a Mario Game where only game scores are available. After 100 episodes adaptation, the EMOA3C agent infers that the underlying preference for the achieving higher score mainly focuses on `x-pos` (0.3725) and `time` (0.2307), which is coincident with the strategy human players commonly use – to achieve higher score, especially the stage accomplishment bonus, the first priority is to ensure Mario can move forward towards the flag within the time limit.